PREMIUM HANDWRITTEN INTERVIEW NOTEBOOK

JAVA MEMORY MODEL

Zero to Hero Interview Guide

See the order. Prove the handoff. Write correct concurrent Java.

80 ORIGINAL QUESTIONS | 80 ANSWERED FOLLOW-UPS | 16 CHAPTERS | 8 JAVA LABS

```
write → release → acquire → read
check ≠ atomic operation
prove correctness → run tests
```

START HERE

Think in threads. Draw every edge.

The complete source question bank rewritten as simple, handwritten-style notes without changing a single question. Every follow-up has its own answer.

80
Unchanged interview questions

80
Individual follow-up answers

08
Testable Java labs

Answer in this order: simple meaning → visual handoff → precise Java guarantee → tiny code trace → interview trap → follow-up answer.

Important distinction: JMM guarantees come from synchronization rules, not from assumptions about CPU cache flushing, thread sleep duration or a passing test.

Verified scope: the original appendix's eight Java sources compile for Java 17 and run locally. The jcstress template requires a separate build. Java 21/25 details are discussion notes rather than independently verified performance tests.

The complete original questions are preserved. Each page includes a source-page reference for tracing the original material.

TABLE OF CONTENTS

16 chapters · foundation to architect

01. Foundations: what the JMM guarantees · Q001–Q005

02. Happens-before rules and proof · Q006–Q010

03. Volatile: power and limitations · Q011–Q015

04. Locks, monitors and conditions · Q016–Q020

05. Safe publication and singleton patterns · Q021–Q025

06. Final fields and genuine immutability · Q026–Q030

07. Atomics, CAS and compound state · Q031–Q035

08. VarHandles, access modes and fences · Q036–Q040

09. Lifecycle, interruption and progress · Q041–Q045

10. Higher-level handoff guarantees · Q046–Q050

11. Collections, aggregates and application state · Q051–Q055

12. Litmus tests and legal outcomes · Q056–Q060

13. Common correctness traps · Q061–Q065

14. Performance, hardware and modern threads · Q066–Q070

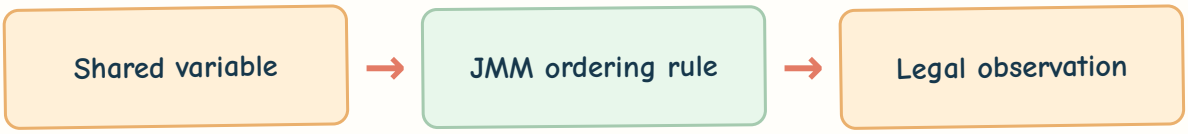
15. Proof, stress testing and evidence · Q071–Q075

16. Senior design and production scenarios · Q076–Q080

CHAPTER 01 / 16

Foundations: what the JMM guarantees

Concurrency is about what threads may observe, not the heap diagram.



FIVE ORIGINAL INTERVIEW QUESTIONS

001. What is the Java Memory Model?

002. How do visibility, atomicity and ordering differ?

003. What is a data race?

004. What is sequential consistency?

005. Does happens-before mean an action happened earlier by the clock?

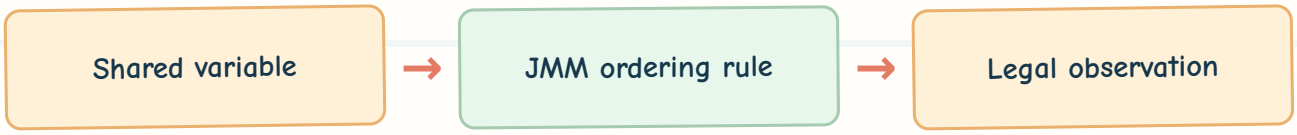
> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q001. What is the Java Memory Model?

SHORT ANSWER · SAY THIS FIRST

> The JMM is Java's rulebook for how threads see shared data and in what order.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

The JMM defines which reads and writes may be observed across threads and the ordering constraints imposed by synchronization. It lets implementations optimize execution while preserving specified behavior. It is a concurrency contract, distinct from a diagram of heap, stacks and metaspace.

JAVA / CONCURRENCY SCENARIO

```
Question: can thread B observe thread A's update?
JMM answer: inspect actions and synchronization edges.
Memory-area answer: where are objects and frames managed?
```

What happens: Visibility and legal observations need a synchronization argument.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why do two machines sometimes show different outcomes for the same racy code?

Answer: Different CPU instructions, compiler optimizations and scheduling can expose different outcomes because a data race lacks a guaranteed ordering. Correctly synchronized code must not rely on one machine's behavior.

Interview trap: Explaining only heap versus stack does not answer a JMM question.

Original source p. 6 · References M01, M21-M23

Q002 · Foundations: what the JMM guarantees

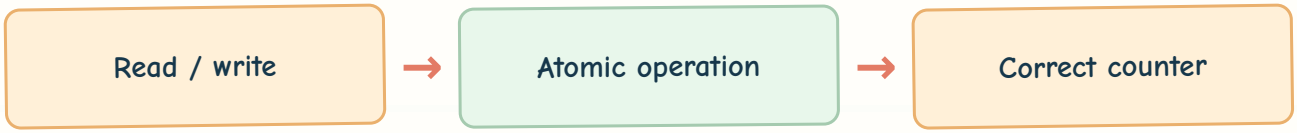
INTERVIEW ANSWER CARD

Q002. How do visibility, atomicity and ordering differ?

SHORT ANSWER · SAY THIS FIRST

> Visibility is seeing changes, atomicity is doing one indivisible step, and ordering says what may happen first.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Visibility concerns whether a read can observe another thread's writes. Atomicity concerns whether an operation is indivisible at the required level. Ordering constrains which observations and action sequences are permitted. One mechanism can provide some guarantees without providing every business invariant.

JAVA / CONCURRENCY SCENARIO

```
volatile int count;
count++; // read, compute and write: not one atomic increment
```

What happens: Volatile access does not combine multiple actions into one transaction.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which mechanism would you choose for an exact concurrent counter?

Answer: Use AtomicInteger.incrementAndGet() for a single exact counter, or a lock if the count must change together with other state.

Interview trap: 'Thread-safe' without naming the operation or invariant is imprecise.

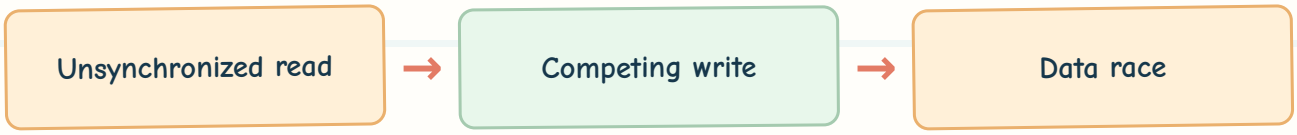
Original source p. 6 · References M01, M21-M23

Q003. What is a data race?

SHORT ANSWER · SAY THIS FIRST

A data race happens when threads access the same data without an ordering rule and at least one writes.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Conflicting accesses to the same variable form a data race when at least one is a write and they are not ordered by happens-before. Data-race freedom is a property of the program's synchronization, not of whether a failure happened during a test.

JAVA / CONCURRENCY SCENARIO

```
int balance; // shared ordinary field
A: balance = 100;
B: int seen = balance; // no connecting synchronization
```

What happens: The accesses lack the required ordering even if the hardware usually shows 100.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Can a program using only atomic accesses still violate a business rule?

Answer: Yes. Two separate atomic reads or updates can be individually safe while the combined check and business decision is wrong; protect the whole rule.

Interview trap: A data race and a broader business race condition are related but not identical terms.

Original source p. 7 · References M01, M21-M23

Q004 · Foundations: what the JMM guarantees INTERVIEW ANSWER CARD

Q004. What is sequential consistency?

SHORT ANSWER · SAY THIS FIRST

> Sequential consistency means actions look like one shared timeline that keeps each thread's own order.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Sequential consistency means observations can be explained by one interleaving that respects each thread's program order. Correctly synchronized, data-race-free Java programs receive this guarantee under the JMM's conditions. Racy ordinary code is not generally entitled to that simple interleaving model.

JAVA / CONCURRENCY SCENARIO

```
A: write x; read y
B: write y; read x
Ask whether one order can explain all observations.
```

What happens: Synchronization makes reasoning closer to interleaving source-level actions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why can a data-race-free check-then-act workflow still be wrong?

Answer: Because another thread can change the state after a safe read but before a safe action. The check and action must form one atomic step.

Interview trap: Sequential consistency does not make two separately synchronized fields one atomic snapshot.

Original source p. 7 · References M01, M21-M23

Q005 · Foundations: what the JMM guarantees

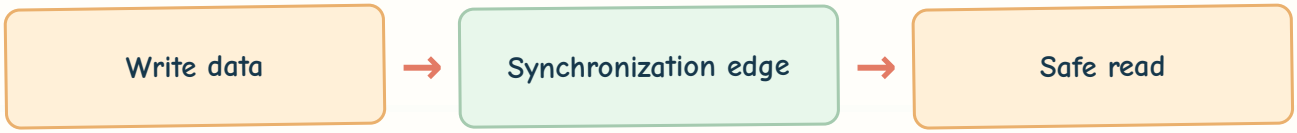
INTERVIEW ANSWER CARD

Q005. Does happens-before mean an action happened earlier by the clock?

SHORT ANSWER · SAY THIS FIRST

Happens-before is a Java ordering promise, not simply one event occurring earlier on a clock.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. Happens-before is a specified ordering relation used to constrain observations. Clock time alone does not create it. Nor does an edge necessarily mean every later read sees that exact write if other allowed writes intervene.

JAVA / CONCURRENCY SCENARIO

```
A writes data at 10:00:00
B reads data at 10:00:01
Elapsed time alone supplies no publication guarantee.
```

What happens: Prove an edge through synchronization rather than through timing.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What would establish a reliable edge between those actions?

Answer: Use a matching volatile write/read, unlock/lock on the same monitor, a queue handoff or another documented synchronizing action.

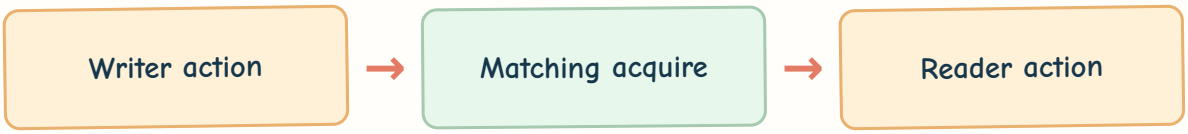
Interview trap: Sleeps, logs and elapsed delays are not substitutes for the required memory contract.

Original source p. 7 · References M01, M21-M23

CHAPTER 02 / 16

Happens-before rules and proof

Make every cross-thread arrow represent a real Java rule.



FIVE ORIGINAL INTERVIEW QUESTIONS

006. What are the most useful happens-before rules?

007. How does transitivity make publication work?

008. What does the thread-start rule guarantee?

009. What does join guarantee?

010. Why does using the same monitor matter?

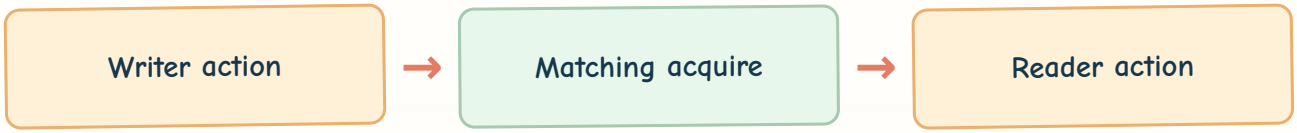
> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q006. What are the most useful happens-before rules?

SHORT ANSWER · SAY THIS FIRST

Useful rules include program order, volatile handoff, lock release/acquisition and thread start or completion.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Program order supplies within-thread edges. Monitor unlock precedes a subsequent lock of the same monitor. Volatile write precedes subsequent read of that variable in synchronization order. Thread start, termination detection and documented concurrent APIs supply additional edges. Transitivity connects them.

JAVA / CONCURRENCY SCENARIO

```
Write data -> publish -> acquire -> read data
Every arrow needs a rule or documented API contract.
```

What happens: A proof names the exact variable, monitor or handoff operation.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which edge is missing in an unsynchronized getter?

Answer: The ordinary getter has no matching acquire action, so it lacks an edge connecting the writer's updates to its read.

Interview trap: 'There is a lock somewhere' does not prove an edge for this reader.

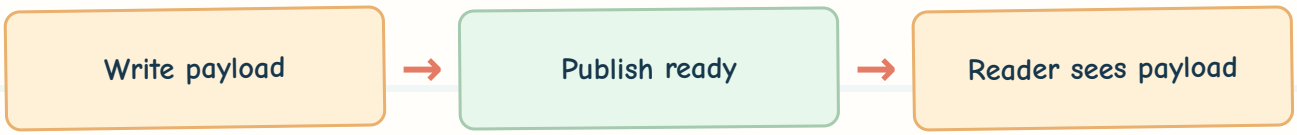
Original source p. 8 · References M01, M06, M07

Q007. How does transitivity make publication work?

SHORT ANSWER · SAY THIS FIRST

If A happens-before B and B happens-before C, then A happens-before C too.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

If A happens-before B and B happens-before C, A happens-before C. Ordinary writes before a publication action can therefore become ordered before reads after a corresponding acquiring action. The handoff must actually participate in the protocol.

JAVA / CONCURRENCY SCENARIO

```
Writer: payload = 42; ready = true; // ready is volatile
Reader: if (ready) use(payload);
Chain: payload write -> ready write -> read -> payload read
```

What happens: The ordinary payload access is protected by the publication chain in this one-shot protocol.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you design a reusable message channel?

Answer: Use BlockingQueue, another documented channel or a versioned immutable message with an atomic publication step; define ownership and reuse.

Interview trap: Resetting the flag and reusing payload introduces lifecycle questions that this example does not solve.

Original source p. 9 · References M01, M06, M07

Q008. What does the thread-start rule guarantee?

SHORT ANSWER · SAY THIS FIRST

> Writes made before Thread.start() are visible to the new thread through that start handoff.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Actions in the thread that precede starting another thread are ordered before actions in the started thread. Construct and configure the task before start. The rule does not publish arbitrary later mutations back and forth automatically.

JAVA / CONCURRENCY SCENARIO

```
int[] data = {42};
Thread worker = new Thread(() -> use(data[0]));
worker.start();
```

What happens: The prepared array value is handed to the worker through start ordering.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What if the parent changes data[0] after start?

Answer: Later changes are not covered by start; use a new synchronization mechanism or transfer immutable data rather than racing on the array.

Interview trap: Creating a Thread object without starting it is not the same edge.

Original source p. 9 · References M01, M06, M07

Q009 · Happens-before rules and proof

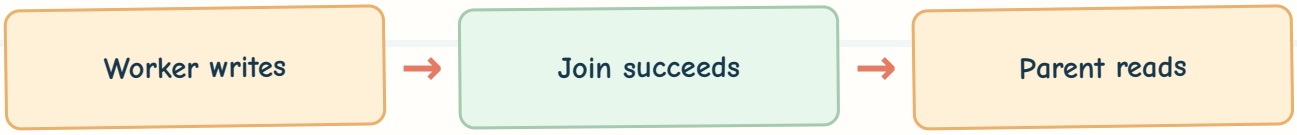
INTERVIEW ANSWER CARD

Q009. What does join guarantee?

SHORT ANSWER · SAY THIS FIRST

After a successful join detects completion, the parent may safely read the worker's completed writes.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

When a thread's termination is detected through a successful join, its completed actions are ordered before the joining thread continues with dependent reads. A timed join that returns while the target is still alive does not establish completion.

JAVA / CONCURRENCY SCENARIO

```
worker.start();
worker.join(); // completes after target termination
use(resultWrittenByWorker);
```

What happens: The completion boundary permits reading results written by that worker.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you verify completion after a timed join?

Answer: After a timed join, check worker.isAlive(); if still alive, do not treat results as complete. Wait or use a Future with a clear completion contract.

Interview trap: A timeout return is not proof that the worker's result is ready.

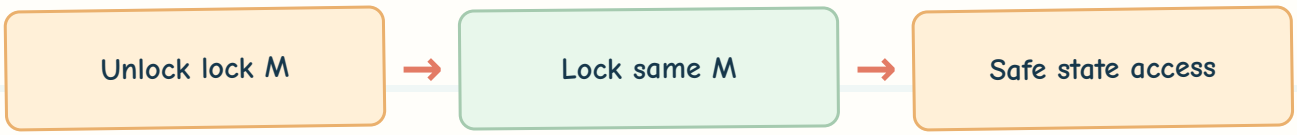
Original source p. 9 · References M01, M06, M07

Q010. Why does using the same monitor matter?

SHORT ANSWER · SAY THIS FIRST

Both sides must use the same lock object for a monitor handoff to work.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Unlock/lock ordering is associated with the same monitor identity. Two different lock objects do not create that synchronization edge. Instance synchronized methods use their receiver; static synchronized methods use the class's monitor.

JAVA / CONCURRENCY SCENARIO

```
A: synchronized (lockA) { state = 1; }
B: synchronized (lockB) { use(state); }
// Different monitor identities do not protect this handoff.
```

What happens: The shared-state policy must identify which lock guards every relevant access.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What documentation would make a locking policy reviewable?

Answer: State in code comments which lock guards each field, who may acquire it, and which reads/writes and invariant must occur under it.

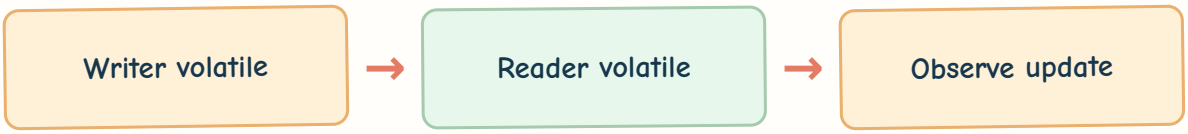
Interview trap: A static synchronized writer and instance synchronized reader use different monitors.

Original source p. 10 · References M01, M06, M07

CHAPTER 03 / 16

Volatile: power and limitations

Visibility is useful; atomic business updates are different.



FIVE ORIGINAL INTERVIEW QUESTIONS

011. What does volatile provide?

012. Why is volatile count++ unsafe for exact counting?

013. Does a volatile reference make the referenced object thread-safe?

014. What is safe one-shot publication with a volatile flag?

015. Is volatile literally a flush to main RAM on every access?

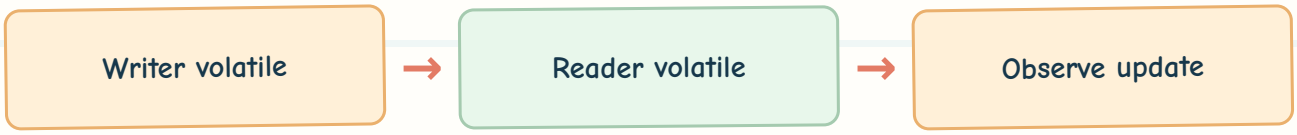
> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q011. What does volatile provide?

SHORT ANSWER · SAY THIS FIRST

> Volatile lets threads see a variable through Java's ordering and visibility rules, but does not lock a multi-step operation.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Volatile reads and writes are atomic accesses with specified ordering and visibility semantics. They participate in synchronization order and can publish earlier ordinary writes through a correct protocol. They do not supply mutual exclusion for a sequence of operations.

JAVA / CONCURRENCY SCENARIO

```
volatile boolean stopped;
void requestStop() { stopped = true; }
boolean stopRequested() { return stopped; }
```

What happens: An eventual worker observation of the flag can coordinate a simple stop request.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would a blocked worker be woken promptly?

Answer: Call interrupt for interruptible waits, notify a guarded condition, or complete an explicit signaling primitive; a flag alone cannot wake blocked I/O.

Interview trap: Visibility is not a fairness or bounded-scheduling guarantee.

Original source p. 11 · References M01, M04, M21

Q012 · Volatile: power and limitations

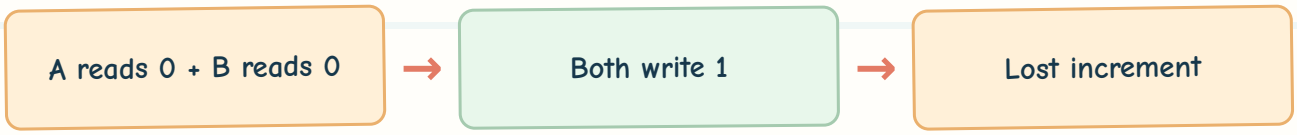
INTERVIEW ANSWER CARD

Q012. Why is volatile count++ unsafe for exact counting?

SHORT ANSWER · SAY THIS FIRST

count++ is read, add and write: two threads can both read 0 and both write 1.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Increment is a read-modify-write sequence. Two threads can both read the same value and then overwrite with the same next value. Each volatile access can be valid while the combined increment loses one logical update.

JAVA / CONCURRENCY SCENARIO

```
Initial count = 0
A reads 0; B reads 0
A writes 1; B writes 1
Final count can be 1, not 2
```

What happens: Protect the compound operation with a suitable atomic update or lock.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why does AtomicInteger.incrementAndGet solve this specific operation?

Answer: incrementAndGet() uses an atomic read-modify-write operation, so concurrent increments on that counter are not lost.

Interview trap: Observing the expected total once does not prove the implementation correct.

Original source p. 11 · References M01, M04, M21

Q013 · Volatile: power and limitations

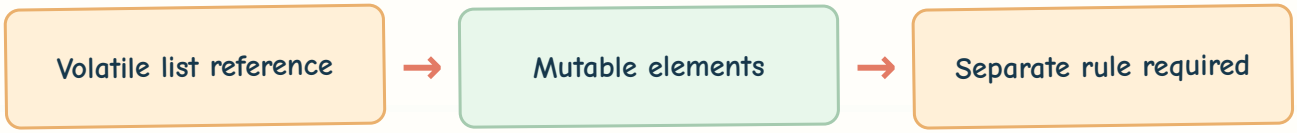
INTERVIEW ANSWER CARD

Q013. Does a volatile reference make the referenced object thread-safe?

SHORT ANSWER · SAY THIS FIRST

Volatile safely publishes a reference but cannot protect changes made inside a mutable object.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. It publishes and synchronizes accesses to the reference. Mutations inside the referenced object need their own discipline unless the object is immutable or safely confined. Replacing a complete immutable snapshot is a common appropriate pattern.

JAVA / CONCURRENCY SCENARIO

```
volatile List<String> names;
names = new ArrayList<>();
names.add("A"); // mutation is not made safe by reference volatility
```

What happens: Reference publication and collection mutation are different actions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you publish a read-only replacement list?

Answer: Create a defensive List.copyOf version and publish it through a volatile reference or AtomicReference; readers keep that immutable snapshot.

Interview trap: A volatile array reference does not turn every element into a volatile variable.

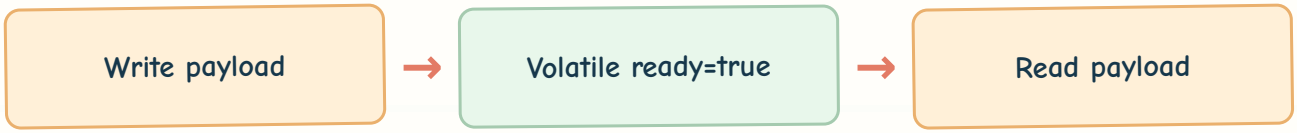
Original source p. 12 · References M01, M04, M21

Q014. What is safe one-shot publication with a volatile flag?

SHORT ANSWER · SAY THIS FIRST

Write the data first, then set a volatile ready flag; the reader checks ready before reading data.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Write all required payload state before writing the volatile ready flag, then read payload only after the reader observes readiness. For a one-shot, single-publication protocol, the chain orders the payload initialization before those reads.

JAVA / CONCURRENCY SCENARIO

```
int payload;
volatile boolean ready;
Writer: payload = 42; ready = true;
Reader: if (ready) assert payload == 42;
```

What happens: The payload can remain ordinary because the protocol supplies ordering.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which outcomes are permitted when the reader sees ready as false?

Answer: If ready is false, it may skip reading and report not-ready, such as -1. It cannot claim that publication was observed.

Interview trap: Reading payload before testing ready, or mutating it later without synchronization, breaks the stated argument.

Original source p. 12 · References M01, M04, M21

Q015 · Volatile: power and limitations

INTERVIEW ANSWER CARD

Q015. Is volatile literally a flush to main RAM on every access?

SHORT ANSWER · SAY THIS FIRST

> Volatile promises ordering and visibility, not a literal RAM flush on every access.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

That is an unreliable implementation explanation. The portable contract is about legal observations and ordering. Compilers and CPUs use architecture-specific instructions, barriers and coherent caches to implement it; Java code should rely on the contract.

JAVA / CONCURRENCY SCENARIO

```
Portable argument: volatile write/read synchronization
Implementation question: inspect generated code and ISA
Neither requires a universal RAM transaction story.
```

What happens: Correctness must not depend on a particular cache layout.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why can the same volatile operation compile differently on x86 and ARM?

Answer: Different processors have different memory-ordering instructions. The JVM uses instructions needed on each platform to implement the same JMM rule.

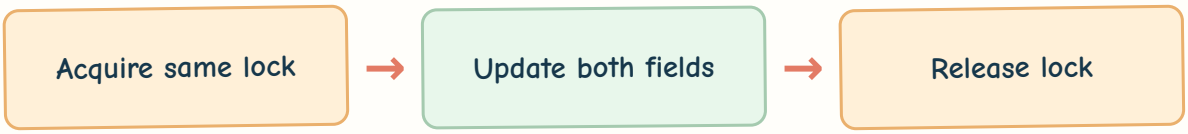
Interview trap: A CPU-cache metaphor can hide the difference between a reference write and a compound invariant.

Original source p. 12 · References M01, M04, M21

CHAPTER 04 / 16

Locks, monitors and conditions

Guard the entire invariant with the same ownership rule.



FIVE ORIGINAL INTERVIEW QUESTIONS

016. What does synchronized guarantee?

017. When should you use ReentrantLock instead of synchronized?

018. Why must wait or await be used in a loop?

019. Does notify release the monitor immediately?

020. Why is locking each getter insufficient for a multi-field snapshot?

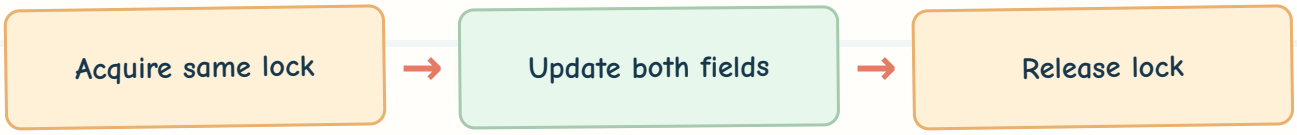
Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q016. What does synchronized guarantee?

SHORT ANSWER · SAY THIS FIRST

Synchronized allows only one thread at a time to hold the same monitor and connects state changes across lock boundaries.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

It provides mutual exclusion on one monitor and release/acquire-style memory ordering through its specified unlock/lock relationship. Every relevant access must follow the same policy. Reentrancy lets the owning thread acquire the same monitor again.

JAVA / CONCURRENCY SCENARIO

```
synchronized (lock) {
 balance -= amount;
 ledger.add(entry);
}
```

What happens: A reader using the same lock can observe a consistent guarded invariant.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you return a safe balance-and-ledger summary?

Answer: Read the balance and ledger inside one synchronized block and return an immutable summary made while holding the lock.

Interview trap: An unsynchronized reader is not protected merely because the writer uses synchronized.

Original source p. 13 · References M08, M09, M24, M34

Q017 · Locks, monitors and conditions

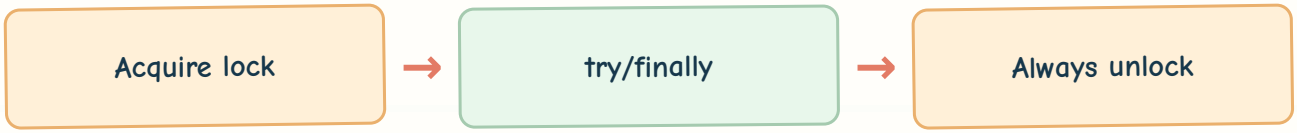
INTERVIEW ANSWER CARD

Q017. When should you use ReentrantLock instead of synchronized?

SHORT ANSWER · SAY THIS FIRST

Use ReentrantLock when timed or interruptible acquisition, conditions or other explicit-lock features are needed.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Use an explicit lock when its supported features, such as interruptible acquisition, timed attempts, fairness options or multiple Conditions, fit the design. The memory contract of successful locking/unlocking parallels monitor guarantees, while lifecycle management requires explicit finally cleanup.

JAVA / CONCURRENCY SCENARIO

```
lock.lock();
try { updateState(); }
finally { lock.unlock(); }
```

What happens: The lock must be released on exceptional control paths.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When would lockInterruptibly be preferable?

Answer: Use lockInterruptibly() when cancellation should stop a thread waiting to acquire a lock, such as a request with a deadline.

Interview trap: Fairness options do not promise a global scheduler or a maximum wait time.

Original source p. 14 · References M08, M09, M24, M34

Q018. Why must wait or await be used in a loop?

SHORT ANSWER · SAY THIS FIRST

Always recheck a waiting condition in a loop because a wakeup is not proof the condition became true.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A wait can return for reasons that do not make the predicate true, including spurious wakeups and another thread consuming the state first. Test the guarded predicate while holding its associated lock, wait, then recheck after reacquisition.

JAVA / CONCURRENCY SCENARIO

```
synchronized (lock) {
 while (!ready) lock.wait();
 consume();
}
```

What happens: Rechecking ties progress to state rather than to the mere occurrence of a notification.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would a timeout change the loop's result contract?

Answer: Compute a deadline and return a distinct timeout result if the predicate is still false after the remaining wait budget is exhausted.

Interview trap: An if check can proceed with a false predicate after waking.

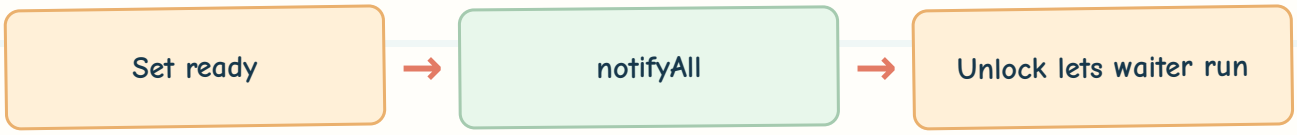
Original source p. 14 · References M08, M09, M24, M34

Q019. Does notify release the monitor immediately?

SHORT ANSWER · SAY THIS FIRST

Notify wakes a waiter, but the notifying thread still holds the monitor until it exits the synchronized block.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. A notified waiter cannot resume the guarded operation until it can reacquire the monitor. Publication is explained through monitor release and reacquisition, with the condition predicate rechecked. Notifications are not durable queued messages by themselves.

JAVA / CONCURRENCY SCENARIO

```
synchronized (lock) {
 ready = true;
 lock.notifyAll();
 // This thread still owns the monitor here.
}
```

What happens: The notifier's later unlock is part of the handoff.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why is the predicate more important than the notification count?

Answer: The boolean/state predicate records the truth; notifications can be missed or spurious. Wait until the predicate is true, not until a signal count matches.

Interview trap: A notification sent before a thread waits can be missed unless the predicate preserves the relevant state.

Original source p. 15 · References M08, M09, M24, M34

Q020 · Locks, monitors and conditions

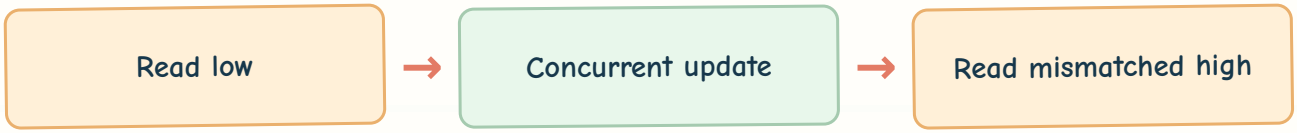
INTERVIEW ANSWER CARD

Q020. Why is locking each getter insufficient for a multi-field snapshot?

SHORT ANSWER · SAY THIS FIRST

Two locked getters can read values from two different moments, giving a broken combined snapshot.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Individually synchronized accesses can be data-race-free yet combine values from different logical versions. Read the whole invariant in one critical section or publish one immutable snapshot. Decide the required atomic observation explicitly.

JAVA / CONCURRENCY SCENARIO

```
int low = getLow();
// Another thread replaces the range here.
int high = getHigh();
// Pair may not represent one committed range.
```

What happens: Atomicity of separate accesses does not imply atomicity of their composition.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What snapshot API would you expose instead?

Answer: Expose one synchronized getSnapshot() that copies all fields under one lock, or return a published immutable record.

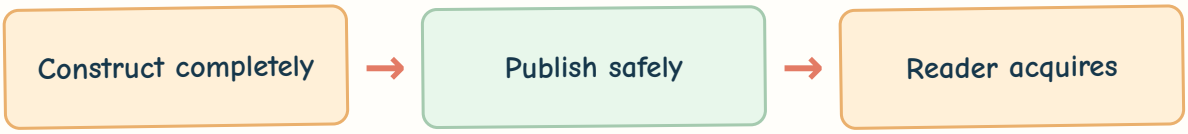
Interview trap: Adding volatile to both fields still does not make a pair-read a transaction.

Original source p. 15 · References M08, M09, M24, M34

CHAPTER 05 / 16

Safe publication and singleton patterns

Finish construction before handing an object to readers.



FIVE ORIGINAL INTERVIEW QUESTIONS

021. What is safe publication?

022. Why can an unsafely published mutable object appear incomplete?

023. Why is double-checked locking correct with a volatile reference?

024. Why does the initialization-on-demand holder idiom work?

025. How do you safely publish configuration updates?

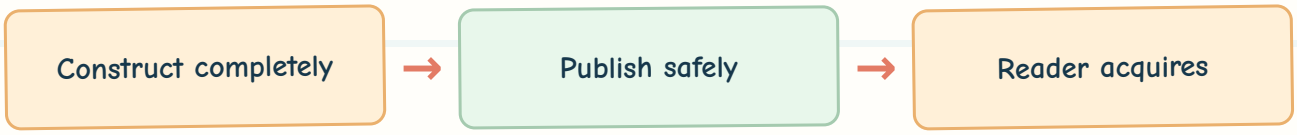
Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q021. What is safe publication?

SHORT ANSWER · SAY THIS FIRST

> Safe publication means an object is fully prepared before another thread gains access through a real synchronization handoff.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Safe publication transfers access to an initialized object through an appropriate ordering mechanism, such as a volatile reference, a shared lock, class initialization or a documented concurrent handoff. Initialization safety and later mutation safety are separate requirements.

JAVA / CONCURRENCY SCENARIO

```
Construct complete state
Publish through a specified handoff
Reader acquires through the matching protocol
```

What happens: The mechanism must cover all state the reader needs.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which concurrent API could publish a work item safely?

Answer: A BlockingQueue.put followed by take safely hands off a prepared work item. Avoid mutating it after transfer without another protocol.

Interview trap: Merely assigning a reference to an ordinary shared field is not a general publication guarantee.

Original source p. 16 · References M03, M15, M21

Q022. Why can an unsafely published mutable object appear incomplete?

SHORT ANSWER · SAY THIS FIRST

A plain shared reference can be seen without a promise that all ordinary fields are initialized for that reader.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Without suitable ordering, another thread that obtains the reference is not generally guaranteed to observe ordinary initialization writes. Source-order intuition alone does not establish the cross-thread relation. Correct final-field initialization has special rules, but is not a cure for arbitrary mutable state.

JAVA / CONCURRENCY SCENARIO

```
class State { int value; State() { value = 42; } }
State shared; // ordinary shared reference
Writer: shared = new State();
Reader: State s = shared; if (s != null) use(s.value);
```

What happens: A non-null reference is not a general proof that every ordinary field is safely observed.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which change supplies publication ordering?

Answer: Make the shared reference volatile, or publish the instance through a synchronized block, concurrent collection or safe class initialization.

Interview trap: Do not describe this as requiring a literal partially allocated object or broken constructor execution.

Original source p. 17 · References M03, M15, M21

Q023 · Safe publication and singleton patterns

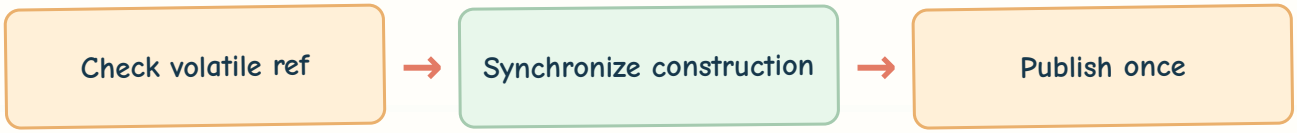
INTERVIEW ANSWER CARD

Q023. Why is double-checked locking correct with a volatile reference?

SHORT ANSWER · SAY THIS FIRST

> Double-checked locking works when the instance reference is volatile and construction is protected by a synchronized slow path.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

The synchronized slow path serializes construction, and the volatile publication orders initialized state before a fast-path reader that acquires the reference. The second null check handles another thread's completed construction. A local variable reduces repeated volatile reads.

JAVA / CONCURRENCY SCENARIO

```
Service s = instance; // instance is volatile
if (s == null) synchronized (Service.class) {
 s = instance;
 if (s == null) instance = s = new Service();
}
return s;
```

What happens: Both publication and singleton construction are covered by distinct mechanisms.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What if the constructor throws before assignment?

Answer: The assignment never runs if construction throws; other callers can retry later. Handle failures explicitly rather than exposing a partial singleton.

Interview trap: Removing volatile leaves the unsynchronized fast path without the required ordinary-state publication guarantee.

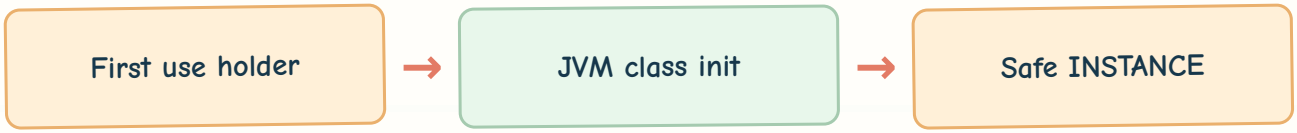
Original source p. 17 · References M03, M15, M21

Q024. Why does the initialization-on-demand holder idiom work?

SHORT ANSWER · SAY THIS FIRST

The holder pattern is safe because Java class initialization publishes the static value correctly.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Class initialization has synchronization and completion rules that safely initialize the static holder state before its valid use. A nested holder defers creation until the holder is initialized. Constructor work and initialization failures still need appropriate handling.

JAVA / CONCURRENCY SCENARIO

```
class Holder {
 static final Service INSTANCE = new Service();
}
Service get() { return Holder.INSTANCE; }
```

What happens: The class-initialization contract supplies the publication boundary.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How does this differ from an eagerly initialized static field?

Answer: An eager static field is created when its declaring class initializes; a nested holder delays construction until the nested holder is first used.

Interview trap: Static means one field per defining class identity, not necessarily one instance across every class loader.

Original source p. 18 · References M03, M15, M21

Q025 · Safe publication and singleton patterns

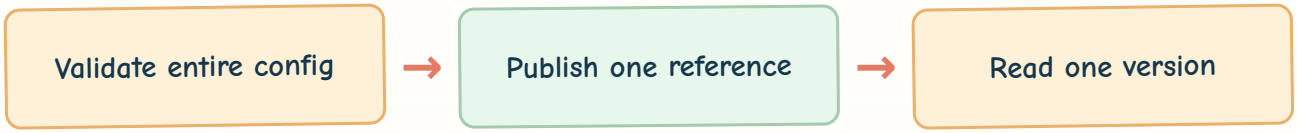
INTERVIEW ANSWER CARD

Q025. How do you safely publish configuration updates?

SHORT ANSWER · SAY THIS FIRST

Publish one complete immutable configuration rather than updating several fields separately.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Build and validate a complete immutable version, then publish one reference through a suitable mechanism. Each reader captures that reference once and uses one version throughout the operation. Coordinate concurrent writers if update composition matters.

JAVA / CONCURRENCY SCENARIO

```
record Config(int timeout, int retries) {}
volatile Config current = new Config(1000, 2);
Config c = current; // capture one version
use(c.timeout(), c.retries());
```

What happens: One reference avoids combining fields from separate updates.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When would an AtomicReference CAS update be needed?

Answer: Use AtomicReference with CAS when several writers must base each change on the current version and must not silently overwrite a concurrent update.

Interview trap: Immutable reads do not prevent two writers from overwriting each other's logical modifications.

Original source p. 18 · References M03, M15, M21

CHAPTER 06 / 16

Final fields and genuine immutability

Final is not deep immutability and does not stop future changes.



FIVE ORIGINAL INTERVIEW QUESTIONS

026. What special guarantee do final fields provide?

027. Does a final reference make its object deeply immutable?

028. Why is this-escape during construction dangerous?

029. What makes an object safely shareable as immutable?

030. Can final-field rules replace all synchronization?

> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q026. What special guarantee do final fields provide?

SHORT ANSWER · SAY THIS FIRST

Final fields of a correctly constructed object get special initialization guarantees, even without a full ordinary publication handoff.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A correctly constructed object that does not escape during construction receives final-field initialization guarantees. When a reader obtains the object, it can observe the properly initialized final values under the specified rules. This special protection is not a general happens-before edge for every ordinary field.

JAVA / CONCURRENCY SCENARIO

```
class Pair {
 final int x;
 int y;
 Pair() { x = 1; y = 2; }
}
```

What happens: Final x and ordinary y have different guarantees under otherwise unsafe publication.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

If a reader sees a non-null Pair, what can it rely on about x versus y?

Answer: For a correctly constructed non-escaping Pair, final x is guaranteed to be initialized as specified; ordinary y is not given the same broad promise by final-field rules.

Interview trap: Do not claim final publishes every field in an arbitrary mutable object.

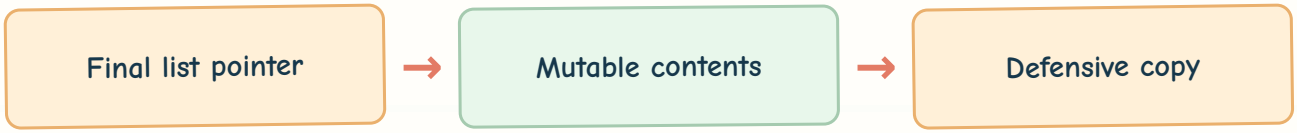
Original source p. 19 · References M02, M21, M23

Q027. Does a final reference make its object deeply immutable?

SHORT ANSWER · SAY THIS FIRST

> Final stops reassignment of the reference; it does not freeze the referenced list or object.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. The final variable cannot be reassigned through ordinary assignment after construction, but the referenced object can still change. Special constructor-time final-field guarantees can include referenced initialization state under the rules; later mutations require separate synchronization.

JAVA / CONCURRENCY SCENARIO

```
final List<String> values = new ArrayList<>();
values.add("A"); // reference remains unchanged
```

What happens: Finality constrains the reference binding, not every reachable object.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would defensive copying change the guarantee?

Answer: Copy mutable inputs and avoid exposing a modifiable original, so external callers cannot mutate the snapshot behind your back.

Interview trap: A record component referencing a mutable list is not automatically deeply immutable.

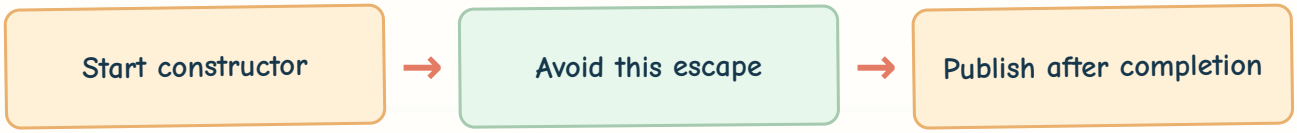
Original source p. 20 · References M02, M21, M23

Q028. Why is this-escape during construction dangerous?

SHORT ANSWER · SAY THIS FIRST

Letting this escape during construction lets another thread call the object before setup is finished.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Publishing this before the constructor finishes can expose incomplete state and defeat the assumptions behind final-field initialization safety. Callback registration, starting work or inserting this into a shared registry can create such escape paths.

JAVA / CONCURRENCY SCENARIO

```
Constructor: registerListener(this);
// Callback may execute before required fields are assigned.
// Prefer a separate post-construction registration step.
```

What happens: The publication timing becomes part of the object's correctness.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would a factory control the lifecycle safely?

Answer: Construct the object privately in a factory, finish its fields and validation, then register/publish it afterward.

Interview trap: Assigning fields before one escape point does not justify every future constructor refactor.

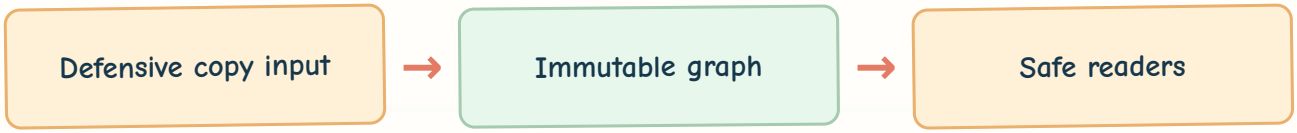
Original source p. 20 · References M02, M21, M23

Q029. What makes an object safely shareable as immutable?

SHORT ANSWER · SAY THIS FIRST

> An immutable object never changes observable state, including the objects it exposes.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Its observable state must not change, including state exposed through reachable mutable components. Complete construction, defensive copies and appropriate APIs establish that property. Choose a publication mechanism whose scope is clear, even when final-field rules offer initialization safety.

JAVA / CONCURRENCY SCENARIO

```
record Snapshot(List<String> values) {
 Snapshot { values = List.copyOf(values); }
}
```

What happens: Copying the list protects its structure, while element immutability must still be considered.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What if each list element is itself mutable?

Answer: If list elements are mutable, List.copyOf only freezes list structure. Deep-copy elements or use immutable element types too.

Interview trap: An unmodifiable view of a mutable original is different from an independent immutable copy.

Original source p. 20 · References M02, M21, M23

Q030 · Final fields and genuine immutability

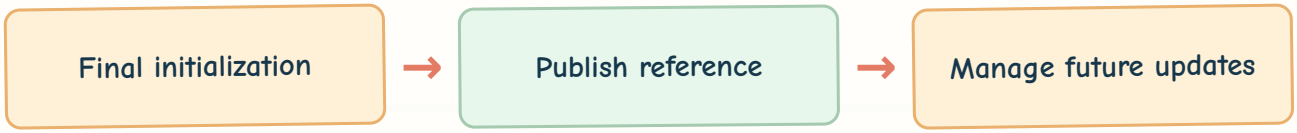
INTERVIEW ANSWER CARD

Q030. Can final-field rules replace all synchronization?

SHORT ANSWER · SAY THIS FIRST

Final-field rules help initial construction, but later changes and cross-thread coordination still need a plan.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. They address specified initialization observations, not coordination, later updates, multi-object invariants or guaranteed reference discovery. Mutable collaboration still needs synchronization. Immutability can remove many data races but not every lifecycle requirement.

JAVA / CONCURRENCY SCENARIO

```
Immutable object: safe read-only state
Shared reference discovery: publication protocol
Later replacement: ordered update mechanism
Business transition: define atomicity separately
```

What happens: A thread may still need a handoff to learn which immutable version to use.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which responsibilities remain in an immutable snapshot service?

Answer: Choose a way to publish new snapshots, serialize or CAS concurrent writers, and specify how readers choose a version.

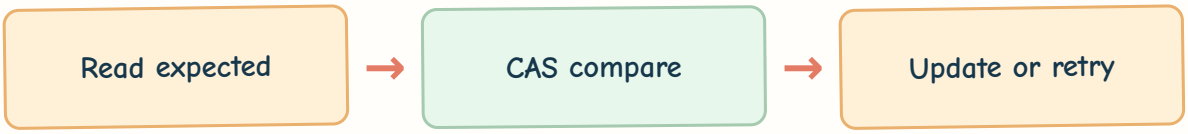
Interview trap: Final does not make a polling loop over an ordinary shared reference a reliable communication channel.

Original source p. 21 · References M02, M21, M23

CHAPTER 07 / 16

Atomics, CAS and compound state

Update one whole state transition, not just one field.



FIVE ORIGINAL INTERVIEW QUESTIONS

031. What is compare-and-set?

032. Why should retrying atomic update functions avoid side effects?

033. How do AtomicInteger and LongAdder differ?

034. What is the ABA problem?

035. How do you update a multi-field invariant atomically?

Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q031 · Atomics, CAS and compound state

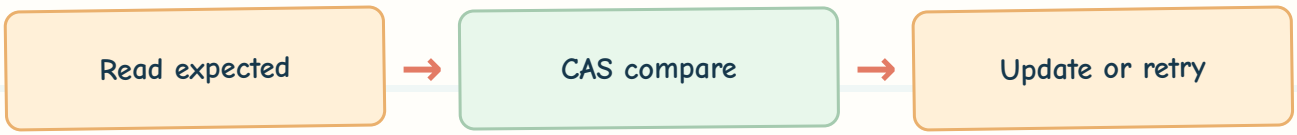
INTERVIEW ANSWER CARD

Q031. What is compare-and-set?

SHORT ANSWER · SAY THIS FIRST

CAS changes one value only when it still equals the expected old value.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

CAS compares the current value with an expected value and atomically replaces it when the comparison matches. The boolean result reports success. Java atomic/VarHandle methods define the applicable access-mode memory effects; not every similarly named weak operation has identical ordering.

JAVA / CONCURRENCY SCENARIO

```
AtomicInteger n = new AtomicInteger(0);
boolean changed = n.compareAndSet(0, 1);
```

What happens: A failed comparison leaves the requested update unapplied.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What state transition can be represented by one atomic reference?

Answer: Store a complete immutable state object or enum in one AtomicReference and CAS that reference from one valid state to another.

Interview trap: CAS is not an arbitrary multi-variable transaction.

Original source p. 22 · References M04, M15-M17, M25

Q032. Why should retrying atomic update functions avoid side effects?

SHORT ANSWER · SAY THIS FIRST

An atomic update function may run more than once, so doing side effects inside it can duplicate them.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

An update function may be invoked repeatedly when contention forces retry. Keep it pure with respect to externally visible effects, and perform separately coordinated side effects after a successful transition where the design allows it.

JAVA / CONCURRENCY SCENARIO

```
state.updateAndGet(old -> computeNext(old));
// Avoid charging a customer inside this retry function.
```

What happens: The successful state update does not imply the computation ran once.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you coordinate a durable event with an in-memory CAS result?

Answer: A local CAS cannot atomically commit an external event; use a durable database transaction and outbox or a coordinated idempotent workflow.

Interview trap: Counting each function invocation as a committed business event can overcount.

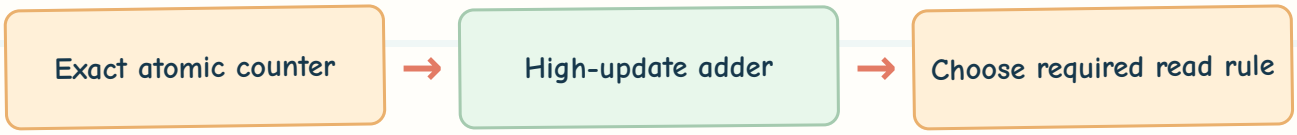
Original source p. 23 · References M04, M15-M17, M25

Q033. How do AtomicInteger and LongAdder differ?

SHORT ANSWER · SAY THIS FIRST

AtomicInteger gives exact updates to one counter; LongAdder scales statistics but concurrent sum is not one atomic snapshot.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

AtomicInteger supports exact atomic operations on one value. LongAdder spreads updates to reduce some contention but its sum is not an atomic snapshot during concurrent updates. It fits statistics better than a strict admission or sequence invariant.

JAVA / CONCURRENCY SCENARIO

```
Exact permit decision -> atomic transition or lock
High-write metric count -> consider LongAdder
Concurrent sum -> aggregate observation, not exact snapshot
```

What happens: The required read semantics decide whether relaxed aggregation is acceptable.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When is an exact post-completion total available?

Answer: Once all writers have finished and their completion is established (for example by join), the final sum may be read as the exact completed total.

Interview trap: A LongAdder total should not decide a strict capacity reservation without another invariant.

Original source p. 23 · References M04, M15-M17, M25

Q034 · Atomics, CAS and compound state

INTERVIEW ANSWER CARD

Q034. What is the ABA problem?

SHORT ANSWER · SAY THIS FIRST

ABA means a value changed from A to B and back to A, fooling a CAS that checks only the value.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A CAS that checks only the expected value can miss that state changed from A to B and back to A. Whether that matters depends on the algorithm. Version/stamp schemes help only when every relevant transition follows the protocol and wraparound/lifecycle are considered.

JAVA / CONCURRENCY SCENARIO

```
Read A
Other actor: A -> B -> A
CAS expects A: value matches despite intervening history
```

What happens: Value equality does not necessarily encode state history.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How does AtomicStampedReference change the comparison?

Answer: AtomicStampedReference checks the expected reference and stamp together, detecting intervening transitions if every writer updates the stamp.

Interview trap: Garbage collection reduces some pointer-reuse concerns but does not eliminate every logical ABA case.

Original source p. 23 · References M04, M15-M17, M25

Q035 · Atomics, CAS and compound state

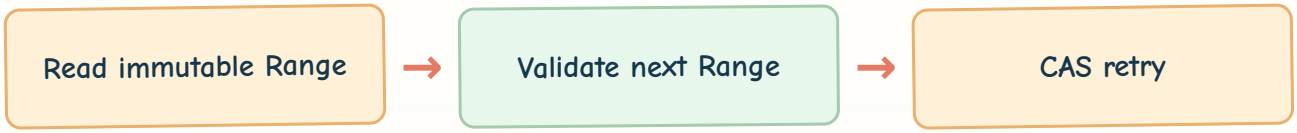
INTERVIEW ANSWER CARD

Q035. How do you update a multi-field invariant atomically?

SHORT ANSWER · SAY THIS FIRST

Update related fields together under one lock or as one immutable object through AtomicReference.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Guard the complete invariant with one lock or represent it as one immutable object updated through an atomic reference. Validate each proposed state. Define how retries and stale expectations are handled rather than splitting related fields into separate atomics.

JAVA / CONCURRENCY SCENARIO

```
record Range(int low, int high) {}
AtomicReference<Range> range =
 new AtomicReference<>(new Range(0, 10));
// CAS a complete valid Range, not two unrelated fields.
```

What happens: A reader captures one coherent version of the invariant.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would two concurrent range edits preserve low <= high?

Answer: Both writers build a whole candidate Range, validate low <= high and retry CAS against the latest range; never update low and high independently.

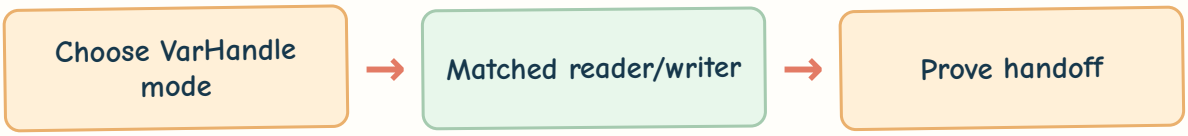
Interview trap: Atomic low and atomic high separately still allow a reader to combine different versions.

Original source p. 24 · References M04, M15-M17, M25

CHAPTER 08 / 16

VarHandles, access modes and fences

Use low-level access modes only when you can prove the handoff.



FIVE ORIGINAL INTERVIEW QUESTIONS

036. What is a VarHandle?

037. How do plain, opaque, acquire/release and volatile modes differ?

038. How does release/acquire publication work?

039. Does adding fullFence make an ordinary shared flag safe?

040. What is the distinction between strong and weak CAS modes?

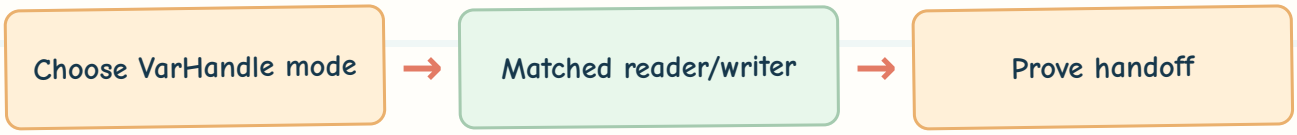
Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q036. What is a VarHandle?

SHORT ANSWER · SAY THIS FIRST

VarHandle gives explicit ways to read, write and atomically change a field with different memory-ordering strengths.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A VarHandle provides typed access to a variable-like location with supported access modes, including ordinary, opaque, acquire/release, volatile and atomic operations. It is an API for explicit memory semantics; use simpler abstractions when they fit the design.

JAVA / CONCURRENCY SCENARIO

```
VarHandle READY = lookup.findVarHandle(
 State.class, "ready", boolean.class);
READY.setRelease(state, true);
```

What happens: The location type and access mode determine supported behavior.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why is mixing plain and stronger accesses risky without a protocol?

Answer: A plain read can bypass the acquire half of a publication protocol. Document which modes every writer and reader must use.

Interview trap: A VarHandle does not automatically make every access to the underlying state synchronized.

Original source p. 25 · References M04, M05, M22

Q037. How do plain, opaque, acquire/release and volatile modes differ?

SHORT ANSWER · SAY THIS FIRST

> Plain, opaque, acquire/release and volatile are increasingly different ordering contracts, not interchangeable performance switches.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Plain accesses provide minimal ordering beyond applicable ordinary semantics. Opaque accesses add coherence for that location with weaker ordering than acquire/release. Acquire/release can constrain surrounding operations in a handoff; volatile provides stronger synchronization-order semantics. Exact method contracts matter.

JAVA / CONCURRENCY SCENARIO

```
Plain: ordinary access discipline
Opaque: coherent location, weak surrounding order
Release/acquire: directed handoff ordering
Volatile: stronger ordering among volatile actions
```

What happens: Choose the weakest mode only when its proof is understood and worth the complexity.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which mode is sufficient for your one-shot payload handoff?

Answer: A release write paired with an acquire read that sees it is sufficient for the described one-shot data handoff; volatile is simpler when unsure.

Interview trap: Opaque is not a drop-in replacement for a volatile publication flag.

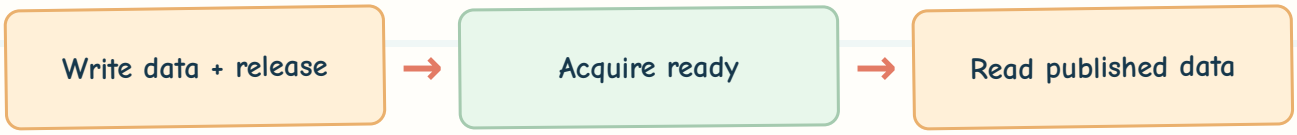
Original source p. 26 · References M04, M05, M22

Q038. How does release/acquire publication work?

SHORT ANSWER · SAY THIS FIRST

Release publishes earlier payload writes; acquire lets a reader safely consume them when it observes the released state.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Write payload before a release store to the handoff variable; read payload only after an acquire load observes the published state under a correct protocol. This orders the earlier payload effects without requiring every payload field to be volatile.

JAVA / CONCURRENCY SCENARIO

```
Writer: state.payload = 42;
        READY.setRelease(state, true);
Reader: if ((boolean) READY.getAcquire(state))
          use(state.payload);
```

What happens: The handoff variable carries ordering for the surrounding accesses.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why does a reusable flag need a stronger lifecycle protocol?

Answer: A reused true/false flag may let a reader confuse old and new messages. Add sequence numbers, per-message ownership or a queue protocol.

Interview trap: An acquire read returning the old false value does not mean publication was acquired.

Original source p. 26 · References M04, M05, M22

Q039 · VarHandles, access modes and fences

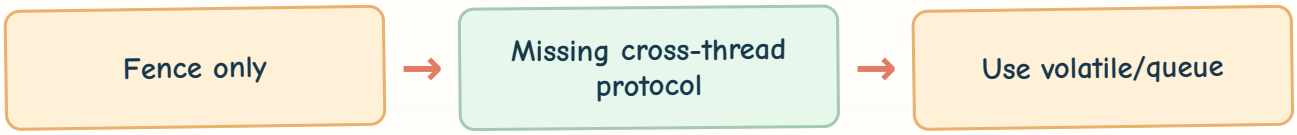
INTERVIEW ANSWER CARD

Q039. Does adding fullFence make an ordinary shared flag safe?

SHORT ANSWER · SAY THIS FIRST

One fence cannot turn an ordinary shared flag into a complete reliable communication protocol.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A fence constrains specified reorderings around that action, but it does not by itself create a complete communication protocol or the same-variable synchronization edge of a volatile flag. Reason about publication and observation together using documented access semantics.

JAVA / CONCURRENCY SCENARIO

```
Write payload; fullFence(); write plain ready
Read plain ready; fullFence(); read payload
// Do not assume this equals a proven volatile protocol.
```

What happens: Local ordering constraints and inter-thread synchronization are related but distinct proof obligations.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What simpler mechanism would you use first?

Answer: Start with a volatile flag or a BlockingQueue, which provides a documented and easier-to-review handoff.

Interview trap: Scattering fences can make code harder to reason about without establishing the required handoff.

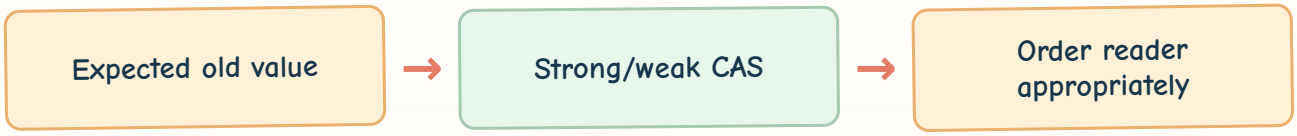
Original source p. 27 · References M04, M05, M22

Q040. What is the distinction between strong and weak CAS modes?

SHORT ANSWER · SAY THIS FIRST

Weak CAS may fail even if the expected value still matches; strong CAS follows its specified comparison contract.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Weak CAS operations can have spurious failure where documented, and VarHandle/atomic variants can have different plain/acquire/release/volatile memory effects. Retry only when the algorithm permits it and choose a mode matching the required read/write ordering.

JAVA / CONCURRENCY SCENARIO

```
Strong CAS: comparison success/failure by contract
Weak mode: may fail spuriously, retry as appropriate
Name suffix: inspect the specified access effects
```

What happens: Failure does not always mean another thread changed the value.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would a release-only successful update affect the following reader?

Answer: A release-only successful update publishes prior writes, but a reader needs a matching acquire (or stronger) read to safely consume them.

Interview trap: The historical weakCompareAndSet naming in atomic APIs should not be interpreted as universally volatile-strength ordering.

Original source p. 27 · References M04, M05, M22

CHAPTER 09 / 16

Lifecycle, interruption and progress

Seeing a stop request does not guarantee progress.



FIVE ORIGINAL INTERVIEW QUESTIONS

041. Do sleep, yield or onSpinWait provide publication?

042. Is interrupt a reliable replacement for a stop flag?

043. Does a timed wait or timed get prove completion?

044. Do LockSupport park/unpark calls provide an entire condition protocol?

045. What is the difference between safety and liveness?

Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q041 · Lifecycle, interruption and progress

INTERVIEW ANSWER CARD

Q041. Do sleep, yield or onSpinWait provide publication?

SHORT ANSWER · SAY THIS FIRST

Sleeping, yielding and spin hints do not publish ordinary shared updates.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. They do not supply a general synchronization edge for ordinary shared state. They influence scheduling or spin-loop execution. A waiting protocol still needs appropriately ordered access and a progress strategy.

JAVA / CONCURRENCY SCENARIO

```
while (!plainReady) Thread.onSpinWait();
// Hint does not repair an ordinary-field data race.
```

What happens: A spin hint can improve implementation behavior without changing the communication contract.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which mechanism would avoid busy waiting here?

Answer: Use a CountDownLatch, BlockingQueue, Condition or Future instead of a polling loop, depending on the lifecycle.

Interview trap: A longer sleep cannot turn unsafe publication into a guaranteed handoff.

Original source p. 28 · References M07, M24, M33

Q042 · Lifecycle, interruption and progress

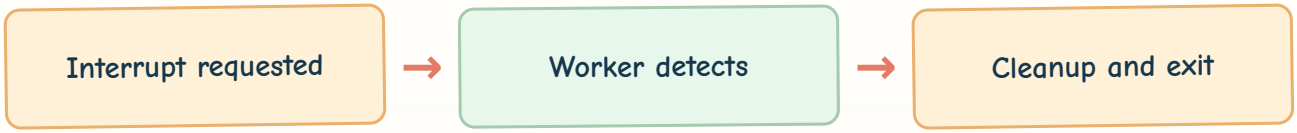
INTERVIEW ANSWER CARD

Q042. Is interrupt a reliable replacement for a stop flag?

SHORT ANSWER · SAY THIS FIRST

> Interrupt is a cancellation request, not a force-stop; code must respond and clean up.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Interruption is a cooperative cancellation signal with specified ordering when detected, but behavior depends on the operation and detection path. Blocking APIs may throw and clear interrupt status. A robust cancellation protocol defines state, wakeup and cleanup rather than treating interruption as forced termination.

JAVA / CONCURRENCY SCENARIO

```
worker.interrupt();
// Worker handles InterruptedException or detects status.
// Preserve or propagate status according to API policy.
```

What happens: Detection participates in the interruption contract; arbitrary code does not automatically stop.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why might you restore status before returning from a helper?

Answer: If a helper catches InterruptedException but cannot handle cancellation fully, call Thread.currentThread().interrupt() so its caller can detect the request.

Interview trap: Swallowing InterruptedException can break the caller's cancellation expectations.

Original source p. 29 · References M07, M24, M33

Q043 · Lifecycle, interruption and progress

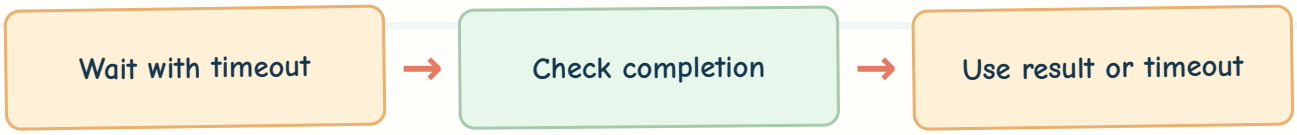
INTERVIEW ANSWER CARD

Q043. Does a timed wait or timed get prove completion?

SHORT ANSWER · SAY THIS FIRST

A timed wait tells you the wait ended; it does not prove the task completed.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. Timeout signals that the requested completion was not obtained within the wait contract. Reads of mutable result state still need the documented completion/publication edge. Cancel or coordinate unfinished work according to policy.

JAVA / CONCURRENCY SCENARIO

```
try { result = future.get(100, MILLISECONDS); }
catch (TimeoutException timeout) { /* not completed */ }
```

What happens: A timeout outcome must be represented separately from a completed result.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What if the task completes just after the timeout?

Answer: Treat the timeout as a separate outcome. The task may complete later, so decide whether to cancel, ignore, or retrieve its result safely.

Interview trap: Proceeding to read ordinary task-written fields after timeout bypasses the intended completion contract.

Original source p. 29 · References M07, M24, M33

Q044 · Lifecycle, interruption and progress

INTERVIEW ANSWER CARD

Q044. Do LockSupport park/unpark calls provide an entire condition protocol?

SHORT ANSWER · SAY THIS FIRST

park/unpark can wake threads, but they do not replace a condition predicate and ordered state.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

They provide permit-based blocking/wakeup facilities, not an automatic complete guarded-state abstraction. Unpark can make a permit available before park, and returns can be spurious. Use correctly ordered state and recheck predicates in a loop.

JAVA / CONCURRENCY SCENARIO

```
while (!orderedCondition()) LockSupport.park();
// Publisher updates ordered state and unparks target.
```

What happens: The application state determines whether progress is allowed.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why are higher-level concurrent utilities often safer?

Answer: Higher-level tools handle queuing, wakeups, permits and failure paths, reducing the number of hand-written race conditions.

Interview trap: A returned park call does not by itself mean the desired state is true.

Original source p. 29 · References M07, M24, M33

Q045 · Lifecycle, interruption and progress

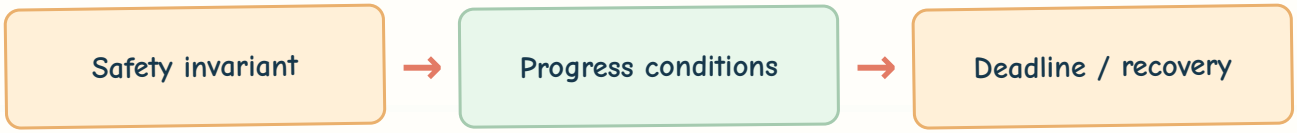
INTERVIEW ANSWER CARD

Q045. What is the difference between safety and liveness?

SHORT ANSWER · SAY THIS FIRST

Safety says bad results never happen; liveness says useful work eventually progresses under stated conditions.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Safety means forbidden outcomes do not happen; liveness means required progress eventually occurs under stated assumptions. Correct visibility does not guarantee scheduling, fairness or successful dependency completion. State both properties in a concurrency design.

JAVA / CONCURRENCY SCENARIO

```
Safety: never publish an invalid range
Liveness: waiting request eventually completes or times out
Resources: threads, queues and locks remain bounded
```

What happens: A perfectly synchronized program can still deadlock or wait forever.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What assumptions does your liveness argument require?

Answer: State fairness/scheduling assumptions, dependency availability, interruption handling, timeouts and resource bounds.

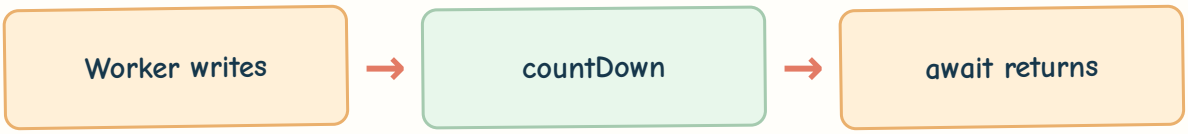
Interview trap: Volatile alone cannot guarantee a worker runs within a deadline.

Original source p. 30 · References M07, M24, M33

CHAPTER 10 / 16

Higher-level handoff guarantees

Let safe concurrency libraries perform the handoffs.



FIVE ORIGINAL INTERVIEW QUESTIONS

046. How does CountDownLatch publish results?

047. How do executor submission and Future.get publish data?

048. What publication does BlockingQueue provide?

049. What do CyclicBarrier and Semaphore guarantee?

050. How should you reason about asynchronous completion chains?

Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

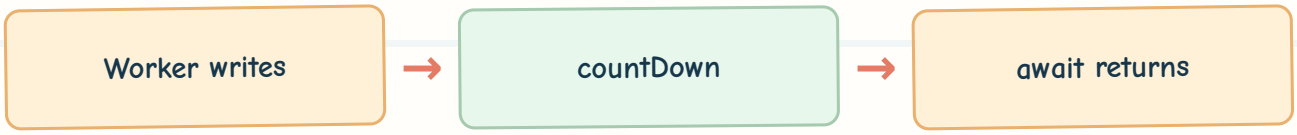
Q046 · Higher-level handoff guarantees INTERVIEW ANSWER CARD

Q046. How does CountDownLatch publish results?

SHORT ANSWER · SAY THIS FIRST

> Writes before countDown are visible after a successful await that crosses the latch's completion.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Actions before countDown are ordered before actions after a corresponding successful await completion. Prepare the result before counting down. A latch is one-shot; timeout without completion is a different result path.

JAVA / CONCURRENCY SCENARIO

```
Worker: result = 42; done.countDown();
Reader: done.await(); use(result);
```

What happens: An ordinary result field can be read through the documented handoff.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which abstraction would you choose for reusable phases?

Answer: Use CyclicBarrier or Phaser for multiple phases, or a fresh latch for each one-shot phase.

Interview trap: Decrementing the latch before writing the result reverses the required order.

Original source p. 31 · References M06, M10-M12, M14, M26-M28

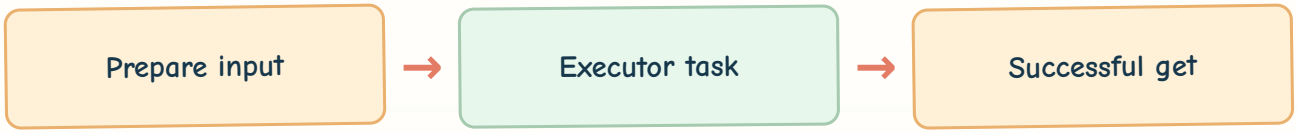
Q047 · Higher-level handoff guarantees INTERVIEW ANSWER CARD

Q047. How do executor submission and Future.get publish data?

SHORT ANSWER · SAY THIS FIRST

Executors publish prepared inputs to tasks, and a completed Future.get publishes task results to callers.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

The executor contract orders pre-submission actions before task execution, and the completion contract orders task actions before following actions after the corresponding get. Use the appropriate successful result path and avoid guessing about unsynchronized completion polls.

JAVA / CONCURRENCY SCENARIO

```
Prepare input -> submit task -> execute
Task writes result -> completed get -> caller reads result
```

What happens: These APIs provide a two-direction handoff for work and results.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you coordinate several task results?

Answer: Submit several tasks, obtain each Future with bounded waits, or use CompletableFuture.allOf and then collect each result safely.

Interview trap: shutdown initiates lifecycle management; it is not a substitute for obtaining the task's result contract.

Original source p. 32 · References M06, M10-M12, M14, M26-M28

Q048 · Higher-level handoff guarantees

INTERVIEW ANSWER CARD

Q048. What publication does BlockingQueue provide?

SHORT ANSWER · SAY THIS FIRST

BlockingQueue safely transfers a prepared element from producer to consumer under its documented handoff.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Actions before placing an element are ordered before another thread's subsequent access/removal of that element under the documented contract. Transfer ownership or use immutable items so later producer mutation does not race with the consumer.

JAVA / CONCURRENCY SCENARIO

```
Message m = new Message(42);
queue.put(m);
Consumer: Message received = queue.take();
```

What happens: The queue safely publishes the prepared message.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you design a retryable work-item lifecycle?

Answer: Give work a stable identifier, define pending/processing/done states, make processing idempotent and retry failed jobs with bounded attempts.

Interview trap: Mutating the same mutable message after put is not covered as an immutable handoff.

Original source p. 32 · References M06, M10-M12, M14, M26-M28

Q049 · Higher-level handoff guarantees

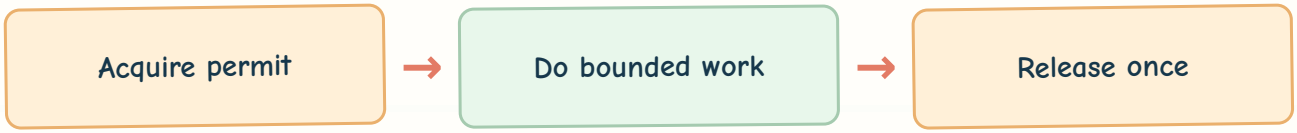
INTERVIEW ANSWER CARD

Q049. What do CyclicBarrier and Semaphore guarantee?

SHORT ANSWER · SAY THIS FIRST

> Barriers coordinate phases; semaphores manage permits. Neither makes a whole business transaction atomic.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

These APIs document memory-consistency effects for their successful coordination operations. A barrier coordinates phases and optionally a barrier action; a semaphore bounds permits and provides release/acquire ordering. Neither makes an arbitrary multi-step business workflow automatically atomic.

JAVA / CONCURRENCY SCENARIO

```
Semaphore: write state -> release -> successful acquire
Barrier: pre-await work -> phase coordination -> post-await
Use the exact API contract for the proof.
```

What happens: The application must align state ownership with the permitted phase or permit.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would exceptions affect permit cleanup?

Answer: Release a permit in finally after successful acquisition, and cover cancellation, rejection and thrown exceptions without double-release.

Interview trap: Releasing permits for the wrong resource lifetime can violate the capacity invariant despite correct memory ordering.

Original source p. 32 · References M06, M10-M12, M14, M26-M28

Q050 · Higher-level handoff guarantees

INTERVIEW ANSWER CARD

Q050. How should you reason about asynchronous completion chains?

SHORT ANSWER · SAY THIS FIRST

An async chain is safe only when each stage has a defined state owner and documented handoff.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Use documented completion and handoff contracts, and define who owns each stage's mutable state. CompletableFuture stages can run on different or completing threads depending on the method. Capture immutable inputs and return results instead of relying on unrelated shared side channels.

JAVA / CONCURRENCY SCENARIO

```
CompletionStage<Result> stage = load()
 .thenApply(input -> transform(input));
// Keep transformation data in the stage result path.
```

What happens: The execution thread is not a reliable implicit synchronization policy.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What happens if two stages mutate the same shared list?

Answer: Two stages adding to a normal shared ArrayList can race or corrupt the result; use stage-local results and combine, or synchronize shared mutation.

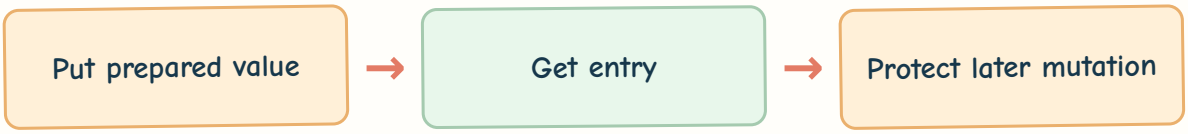
Interview trap: Adding Async changes scheduling choice but does not repair all shared-state races inside callbacks.

Original source p. 33 · References M06, M10-M12, M14, M26-M28

CHAPTER 11 / 16

Collections, aggregates and application state

A thread-safe container does not fix unsafe value objects.



FIVE ORIGINAL INTERVIEW QUESTIONS

051. What does ConcurrentHashMap safely publish?

052. Why is get-then-put unsafe for a single mapping decision?

053. Why is ConcurrentHashMap.size unsuitable for strict admission?

054. What does CopyOnWriteArrayList snapshot iteration mean?

055. Why are Spring singleton beans not automatically thread-safe?

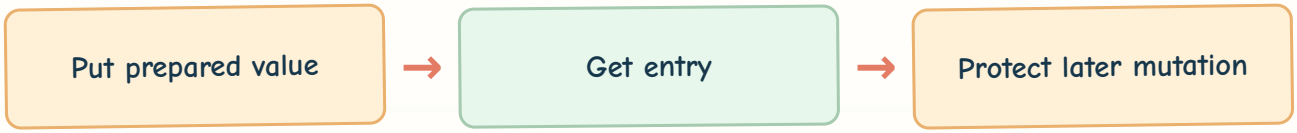
> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q051. What does ConcurrentHashMap safely publish?

SHORT ANSWER · SAY THIS FIRST

> ConcurrentHashMap safely coordinates map entries, not every mutable object stored as a value.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A successful retrieval that reports a particular updated value has a documented happens-before relationship with the associated update. This supports safely accessing prepared values. It does not establish a global atomic snapshot of all mappings or make later value mutations safe.

JAVA / CONCURRENCY SCENARIO

```
map.put("config", preparedSnapshot);
Snapshot s = map.get("config");
// A returned associated value has the per-key contract.
```

What happens: The map coordinates its mapping, not every field lifecycle inside every value.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you handle changing values?

Answer: Store immutable replacement values and use compute or replace-based atomic map updates; do not mutate an ArrayList value concurrently.

Interview trap: A concurrent map of mutable ArrayLists does not make add calls on those lists safe.

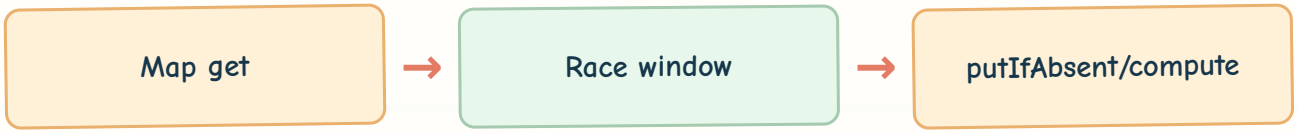
Original source p. 34 · References M13, M30

Q052. Why is get-then-put unsafe for a single mapping decision?

SHORT ANSWER · SAY THIS FIRST

A get followed by put is two steps; another thread may update the key between them.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Another thread can change the mapping between independent operations. Use an appropriate atomic map operation such as putIfAbsent or a documented computation operation. Define the desired behavior when values already exist or computation fails.

JAVA / CONCURRENCY SCENARIO

```
if (map.get(key) == null) map.put(key, value);
// Compare with an appropriate map.putIfAbsent(key, value).
```

What happens: Each map call can be thread-safe while the composition still races.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What side-effect constraints should a mapping computation respect?

Answer: Keep mapping functions short and free of repeatable external side effects; use documented compute rules and avoid recursive updates to the same map.

Interview trap: A concurrent collection does not eliminate every check-then-act bug.

Original source p. 35 · References M13, M30

Q053 · Collections, aggregates and application state

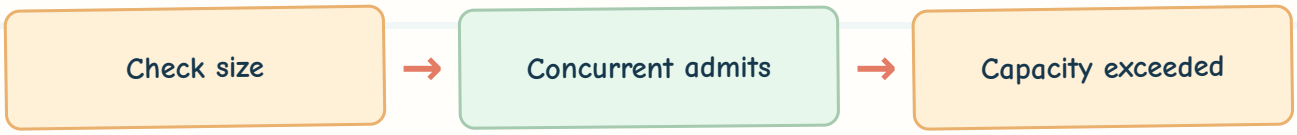
INTERVIEW ANSWER CARD

Q053. Why is ConcurrentHashMap.size unsuitable for strict admission?

SHORT ANSWER · SAY THIS FIRST

> A map's size is an observation, not an atomic reservation of a future slot.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Aggregate observations can reflect concurrent changes without being an atomic control boundary for a later insertion. A strict capacity limit requires a coordinated permit or invariant, not size followed by put. Decide how removals and failures release capacity.

JAVA / CONCURRENCY SCENARIO

```
if (map.size() < LIMIT) map.put(key, value);
// Multiple callers can pass the check concurrently.
```

What happens: The observation and reservation are separate actions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would a semaphore help, and what failure path must release a permit?

Answer: Acquire a semaphore permit before inserting; transfer ownership carefully and release exactly once on removal, rejection or failure.

Interview trap: Replacing size with an atomic counter still needs correct coordination with mapping lifecycle.

Original source p. 35 · References M13, M30

Q054. What does CopyOnWriteArrayList snapshot iteration mean?

SHORT ANSWER · SAY THIS FIRST

CopyOnWriteArrayList iterators see a fixed snapshot, even if the list changes later.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

An iterator observes the array snapshot associated with its creation and does not incorporate later structural changes. Writes copy the backing representation, which can be costly. Mutable elements can still change independently and need their own policy.

JAVA / CONCURRENCY SCENARIO

```
Iterator<Item> it = listeners.iterator();
listeners.add(newListener);
// Existing iterator does not include that structural add.
```

What happens: Stable structure is useful for some read-heavy listener patterns.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When would frequent large-list writes make this unsuitable?

Answer: Every mutation copies the backing array, making large or frequently updated lists expensive in allocation and write time.

Interview trap: Snapshot structure is not deep immutability of every referenced element.

Original source p. 35 · References M13, M30

Q055. Why are Spring singleton beans not automatically thread-safe?

SHORT ANSWER · SAY THIS FIRST

Spring singleton means one bean instance per application context, not one thread per request.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Singleton scope controls instance sharing, not atomic access to mutable fields. Multiple requests can use the same bean concurrently. Prefer immutable/stateless behavior or define a coherent synchronization policy for shared application state.

JAVA / CONCURRENCY SCENARIO

```
@Service class Tracker {
 int count;
 void record() { count++; }
}
```

What happens: A single instance makes the shared counter a concurrency concern.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What changes when several application replicas each have their own instance?

Answer: Each replica has its own bean and memory; local locks do not coordinate stock, reservations or quotas across replicas. Use a shared authority.

Interview trap: Framework dependency injection is not a lock around every service call.

Original source p. 36 · References M13, M30

CHAPTER 12 / 16

Litmus tests and legal outcomes

Classify allowed outcomes before running concurrency tests.



FIVE ORIGINAL INTERVIEW QUESTIONS

056. What can happen in a plain message-passing litmus test?

057. What changes when the message-passing flag is volatile?

058. Can both reads be zero in ordinary store buffering?

059. Why is both-zero forbidden in volatile store buffering?

060. Does racy Java allow arbitrary out-of-thin-air values?

> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q056 · Litmus tests and legal outcomes

INTERVIEW ANSWER CARD

Q056. What can happen in a plain message-passing litmus test?

SHORT ANSWER · SAY THIS FIRST

With ordinary data and ready fields, a reader might see not-ready or see a stale payload after ready.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

With ordinary data and flag fields and no other synchronization, observing the flag does not generally guarantee observing the payload initialization. Classify outcomes from the JMM rather than waiting for one CPU to reproduce them.

JAVA / CONCURRENCY SCENARIO

```
Initial: data = 0; ready = false
A: data = 42; ready = true;
B: if (ready) r = data; else r = -1;
Allowed observations include r = -1, 0 or 42.
```

What happens: The stale payload observation is a permitted unsafe-publication outcome.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How does volatile ready change the classification?

Answer: A volatile ready flag creates the needed handoff when its true read observes the writer's true write; in this one-shot example zero then becomes forbidden.

Interview trap: Failing to observe r=0 during testing does not make it forbidden.

Original source p. 37 · References M01, M19-M23

Q057 · Litmus tests and legal outcomes

INTERVIEW ANSWER CARD

Q057. What changes when the message-passing flag is volatile?

SHORT ANSWER · SAY THIS FIRST

With a volatile ready flag, a true read safely publishes the earlier payload write in this one-shot example.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

For the one-shot protocol, a reader that observes ready=true acquires publication of the preceding data write. It must then read the initialized payload under these assumptions. Seeing false still permits the not-ready result.

JAVA / CONCURRENCY SCENARIO

```
Initial: data = 0; volatile ready = false
A: data = 42; ready = true;
B: if (ready) r = data; else r = -1;
Allowed: -1 or 42; forbidden here: 0.
```

What happens: The proof is the transitive publication chain.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which exact action creates the synchronizes-with edge?

Answer: The volatile write ready=true synchronizes-with the subsequent volatile read of ready that observes the publication in the protocol.

Interview trap: Reusing the fields or adding later payload writes changes the assumptions.

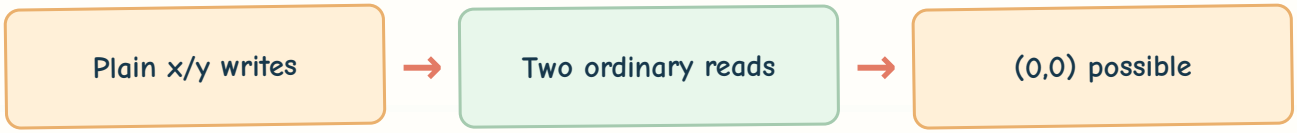
Original source p. 37 · References M01, M19-M23

Q058. Can both reads be zero in ordinary store buffering?

SHORT ANSWER · SAY THIS FIRST

Two ordinary threads can each write one variable and still read zero from the other, because there is no cross-thread order.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Yes, for the ordinary-field test without coordinating synchronization, (0,0) is a permitted result even though no simple source-order sequential interleaving explains it. Writes and reads lack the required cross-thread ordering.

JAVA / CONCURRENCY SCENARIO

```
Initial x = y = 0
A: x = 1; r1 = y;
B: y = 1; r2 = x;
Ordinary x/y: (r1,r2) = (0,0) is allowed.
```

What happens: This illustrates why racy code cannot always be reasoned about as a single source-level interleaving.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What changes if both x and y are volatile?

Answer: If both locations are volatile under the stated store-buffering test, the both-zero outcome becomes forbidden by volatile synchronization order.

Interview trap: join after the actors finish publishes results to the observer but does not retroactively order the actor accesses.

Original source p. 38 · References M01, M19-M23

Q059 · Litmus tests and legal outcomes

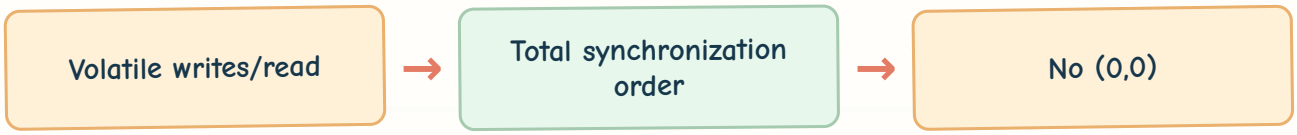
INTERVIEW ANSWER CARD

Q059. Why is both-zero forbidden in volatile store buffering?

SHORT ANSWER · SAY THIS FIRST

Both zero is forbidden when both variables are volatile because it would contradict one total volatile synchronization order.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Volatile actions participate in a synchronization order consistent with program order. With both locations volatile, explaining two initial-value reads after each thread's own write would require an impossible cycle in that order. The other listed outcomes remain permitted.

JAVA / CONCURRENCY SCENARIO

```
Initial volatile x = y = 0
A: x = 1; r1 = y;
B: y = 1; r2 = x;
Allowed: (0,1), (1,0), (1,1); forbidden: (0,0).
```

What happens: This is stronger than claiming only a simple payload visibility edge.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you document these outcomes in jcstress?

Answer: List accepted outcomes (0,1), (1,0), (1,1) and forbidden (0,0) in jcstress before running; record the exact actors and initial state.

Interview trap: A particular machine's failure to produce both-zero with plain fields does not prove plain and volatile equivalent.

Original source p. 38 · References M01, M19-M23

Q060 · Litmus tests and legal outcomes

INTERVIEW ANSWER CARD

Q060. Does racy Java allow arbitrary out-of-thin-air values?

SHORT ANSWER · SAY THIS FIRST

> Racy Java has weak ordering, but that does not mean every imaginable value may appear.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. The JMM includes causality constraints beyond a simplistic 'anything can happen' explanation. Analyze the exact program, initialization and permitted writes. Avoid treating either one observed interleaving or arbitrary invented values as a complete model.

JAVA / CONCURRENCY SCENARIO

```
Ordinary race: do not promise sequential consistency
Causality: not permission for arbitrary fabricated data
Test-specific outcome table: justify each category
```

What happens: Some formal cases are subtle and deserve specification-level analysis.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What extra assumptions must you state before declaring an outcome impossible?

Answer: State exact initial values, all possible writes, object construction and JMM causality assumptions before ruling out an outcome.

Interview trap: Using C/C++ undefined-behavior slogans as the whole explanation for Java races is misleading.

Original source p. 39 · References M01, M19-M23

CHAPTER 13 / 16

Common correctness traps

Watch for accidental races hidden by familiar APIs.



FIVE ORIGINAL INTERVIEW QUESTIONS

061. Are all Java field reads and writes atomic?

062. Why do println-based experiments mislead?

063. Why does ThreadLocal not solve shared-object concurrency?

064. Why is publishing an array reference not enough for later element updates?

065. Why is a local cache of a shared reference useful but not magic?

> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q061 · Common correctness traps

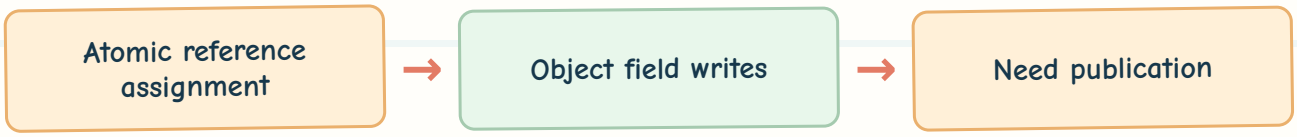
INTERVIEW ANSWER CARD

Q061. Are all Java field reads and writes atomic?

SHORT ANSWER · SAY THIS FIRST

Ordinary reference and many field accesses are atomic, but that says nothing about safe publication of the object's fields.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Reference accesses and relevant primitive single-variable accesses have specified atomicity. The specification permits tearing of nonvolatile long/double accesses; volatile long/double accesses are atomic. Modern implementations may be stronger, but portable reasoning follows the contract.

JAVA / CONCURRENCY SCENARIO

```
Ordinary long/double: specification permits tearing
Volatile long/double: atomic access
Any count++: still a compound operation
```

What happens: Single-variable atomic access and safe publication are different properties.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why does atomic reference assignment not safely publish all mutable fields by itself?

Answer: Reference assignment may be atomic while the referenced mutable object's ordinary initialization is not safely ordered; use volatile or another handoff.

Interview trap: A 64-bit JVM observation does not prove a language-level universal guarantee.

Original source p. 40 · References M01, M05, M29

Q062 · Common correctness traps

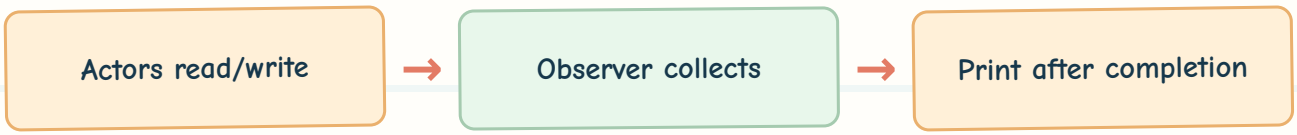
INTERVIEW ANSWER CARD

Q062. Why do println-based experiments mislead?

SHORT ANSWER · SAY THIS FIRST

println can add locking and timing that hide the race you wanted to observe.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Printing can add synchronization, timing changes and optimization effects unrelated to the intended test. It may mask a race or alter its frequency. Use a structured concurrency test and separate result reporting from the actor operations.

JAVA / CONCURRENCY SCENARIO

```
Actor critical path: read/write only intended state
Observer: report completed outcomes afterward
Avoid: logging inside the handoff as a repair
```

What happens: Instrumentation can change the program being investigated.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you collect results without logging in the actors?

Answer: Let actors only perform the tested accesses. The test harness gathers results after completion, classifies outcomes and prints outside actor code.

Interview trap: Incidental synchronization is not a robust application protocol to depend on.

Original source p. 40 · References M01, M05, M29

Q063 · Common correctness traps

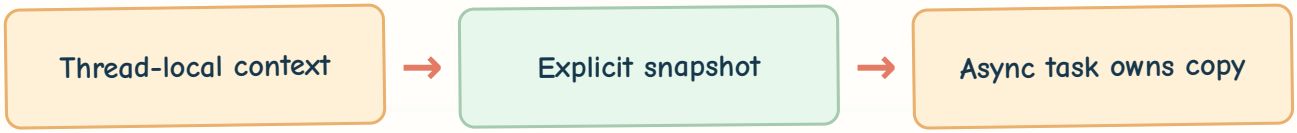
INTERVIEW ANSWER CARD

Q063. Why does ThreadLocal not solve shared-object concurrency?

SHORT ANSWER · SAY THIS FIRST

> ThreadLocal gives each thread its own value; it cannot make some separately shared object thread-safe.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

ThreadLocal associates values with a thread; it does not make the referenced object private if another thread can also access it. Request context and cleanup need explicit ownership. Asynchronous work can move execution to another thread with a different binding.

JAVA / CONCURRENCY SCENARIO

```
ThreadLocal<List<String>> local = new ThreadLocal<>();
local.set(sharedList);
// sharedList remains shared if others hold it.
```

What happens: A thread-specific reference is different from a thread-confined object graph.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you transfer an immutable context to an async task?

Answer: Capture an immutable context and pass it explicitly as a task input, or use a documented context propagation facility with cleanup.

Interview trap: Pool threads can retain request context unless scoped values are removed appropriately.

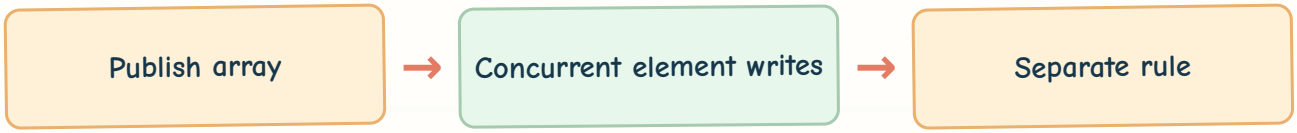
Original source p. 41 · References M01, M05, M29

Q064. Why is publishing an array reference not enough for later element updates?

SHORT ANSWER · SAY THIS FIRST

> Publishing an array reference does not make later unsynchronized element writes safe.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Publication can order the elements prepared before the handoff. Subsequent concurrent changes to ordinary elements need separate synchronization. A volatile array reference changes accesses to that reference, not the element-access mode.

JAVA / CONCURRENCY SCENARIO

```
volatile int[] values = new int[2];
values[0] = 42; // ordinary element mutation
// Replacing the whole immutable array is a different design.
```

What happens: The location being accessed determines the relevant contract.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When would AtomicIntegerArray be suitable?

Answer: Use AtomicIntegerArray when individual indices need atomic access and update; use a lock or immutable array snapshot for whole-array rules.

Interview trap: Reading the volatile reference before every element access does not automatically order an uncoordinated later writer's element update.

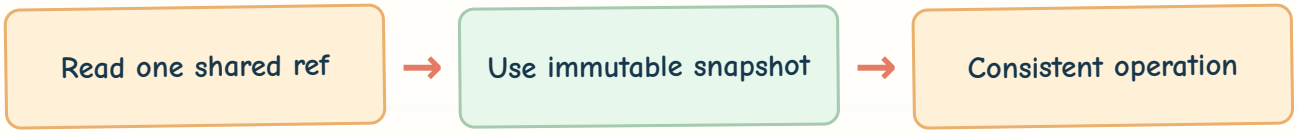
Original source p. 41 · References M01, M05, M29

Q065. Why is a local cache of a shared reference useful but not magic?

SHORT ANSWER · SAY THIS FIRST

> Capture a shared immutable reference once when one operation must use one consistent version.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Capturing an appropriately published reference once can ensure an operation uses one snapshot and avoid repeated access cost. It cannot repair unsafe publication or make the referenced object immutable. Choose the state/version semantics first.

JAVA / CONCURRENCY SCENARIO

```
Config c = current; // current is safely published
validate(c); executeUsing(c);
// Both steps use the same captured version.
```

What happens: Version consistency within one operation can be easier to defend.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What if a configuration must stay current throughout a long operation?

Answer: If the operation needs changing configuration mid-flight, define a deliberate refresh point, version check or restart rule; don't silently mix versions.

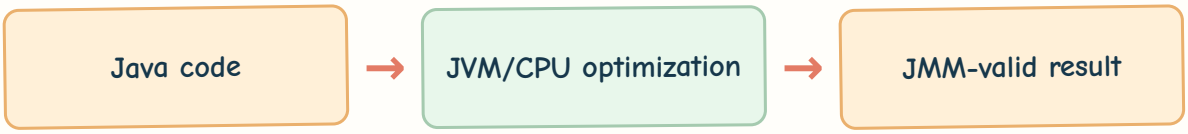
Interview trap: A local variable copied from an ordinary racy field does not create an acquire edge.

Original source p. 41 · References M01, M05, M29

CHAPTER 14 / 16

Performance, hardware and modern threads

Measure before replacing correct, simple synchronization.



FIVE ORIGINAL INTERVIEW QUESTIONS

066. How do compiler and CPU reordering relate to the JMM?

067. What is false sharing?

068. Is lock-free always faster or wait-free?

069. Do virtual threads change JMM synchronization rules?

070. How should you choose a synchronization mechanism for performance?

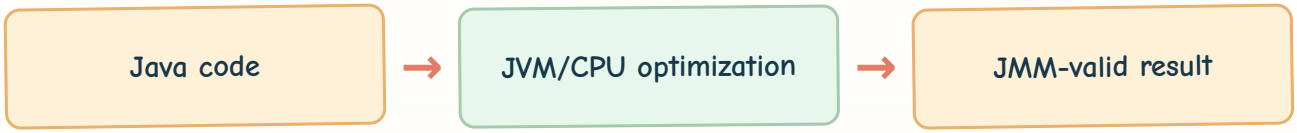
> Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q066. How do compiler and CPU reordering relate to the JMM?

SHORT ANSWER · SAY THIS FIRST

> JIT and CPU may reorder implementation steps as long as Java's permitted observations are preserved.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

The implementation can transform execution when resulting observations satisfy the applicable Java contract. Source code is not a literal assembly schedule. Data-race freedom and synchronization constrain what transformations can be observable to other threads.

JAVA / CONCURRENCY SCENARIO

```
Source -> optimized intermediate form -> machine code
Architecture and access mode affect implementation
Correctness proof -> JMM and API contract
```

What happens: The JMM permits optimization without making every implementation use identical instructions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why can an x86-only test miss a portability issue?

Answer: x86 ordering may accidentally hide a missing Java synchronization edge; another architecture or compiler optimization can expose it.

Interview trap: 'The compiler must execute every line in this order' is not a sound cross-thread proof.

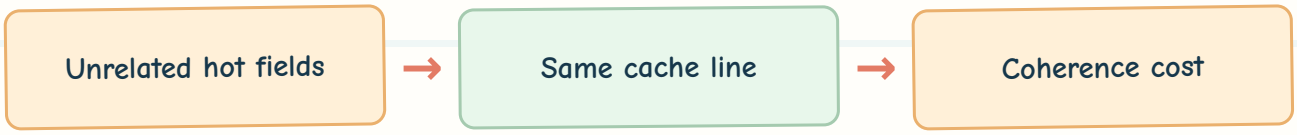
Original source p. 42 · References M22, M31, M32

Q067. What is false sharing?

SHORT ANSWER · SAY THIS FIRST

False sharing occurs when independent hot variables occupy the same CPU cache line and interfere with cache traffic.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Independent writable variables can share a hardware cache line and interfere through coherence traffic even when the Java program has no data race. It is a performance issue rather than a correctness requirement to add volatile everywhere. Verify with representative profiling and benchmarks.

JAVA / CONCURRENCY SCENARIO

```
Thread A repeatedly updates its counter
Thread B repeatedly updates a different nearby counter
Coherence traffic can create performance interference
```

What happens: Logical independence does not imply physical cache-line independence.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What would you measure before introducing padding or layout-specific controls?

Answer: Measure throughput, hardware counters or profiler evidence and compare layout variants; avoid padding without a proven bottleneck.

Interview trap: False sharing and unsafe publication are different diagnoses.

Original source p. 42 · References M22, M31, M32

Q068 · Performance, hardware and modern threads

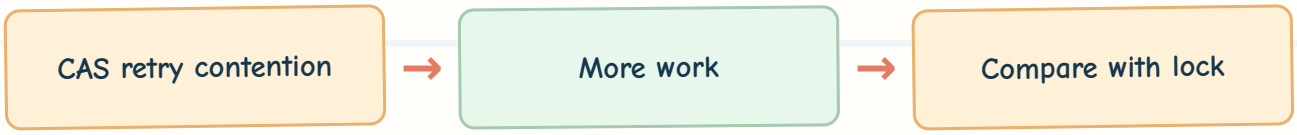
INTERVIEW ANSWER CARD

Q068. Is lock-free always faster or wait-free?

SHORT ANSWER · SAY THIS FIRST

> Lock-free means some thread makes progress; it does not mean everyone is wait-free or faster.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. Lock-free concerns system-wide progress under its algorithmic definition, while wait-free concerns bounded per-operation progress. CAS retries, allocation and contention can make an algorithm slower than a lock for a workload. Java API method names alone do not prove an algorithm's progress class.

JAVA / CONCURRENCY SCENARIO

```
Contention -> failed CAS -> retry cost
Lock-free system progress != every caller bounded latency
Choose: measured workload and correctness requirements
```

What happens: A progress property is not an unconditional speed claim.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

When might a simple lock improve tail latency?

Answer: A short, uncontended lock may reduce retries and CPU use, improving p99 under contention; test the same workload.

> **Interview trap:** A loop around CAS can starve an individual caller.

Original source p. 43 · References M22, M31, M32

Q069 · Performance, hardware and modern threads

INTERVIEW ANSWER CARD

Q069. Do virtual threads change JMM synchronization rules?

SHORT ANSWER · SAY THIS FIRST

Virtual threads share Java's memory rules with platform threads, even though scheduling is different.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. Volatile, locks, atomics and safe-publication rules remain necessary. Virtual threads change execution and waiting economics, not shared-state semantics. Final virtual threads arrived in Java 21; monitor-related carrier pinning improved in Java 24, with runtime-specific caveats.

JAVA / CONCURRENCY SCENARIO

```
Virtual thread A -> shared mutable bean
Virtual thread B -> same shared mutable bean
Same invariant and synchronization proof still required
```

What happens: Cheaper waiting can increase admitted concurrency and expose more shared-state pressure.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you bound downstream work while preserving concurrency safety?

Answer: Use a Semaphore, bounded executor or rate limiter around scarce downstream DB/HTTP capacity, and safely publish shared data as before.

Interview trap: Virtual threads do not make a non-atomic increment correct.

Original source p. 43 · References M22, M31, M32

Q070. How should you choose a synchronization mechanism for performance?

SHORT ANSWER · SAY THIS FIRST

> Choose the simplest mechanism that proves the required invariant, then measure cost and latency.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Start with a clear invariant and the simplest correct mechanism. Measure contention, useful throughput, latency and allocation under representative resources. Use weaker memory modes only with a complete proof and a demonstrated benefit; retain a maintainable ownership model.

JAVA / CONCURRENCY SCENARIO

```
Correct baseline -> profile -> controlled alternative
Compare successful outcomes, p99, CPU and memory
Record runtime, architecture and workload
```

What happens: A faster incorrect execution is not an optimization.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which result would make you keep the simpler locked design?

Answer: Keep a lock if it meets throughput and p99 targets with lower complexity and fewer errors than the lock-free alternative.

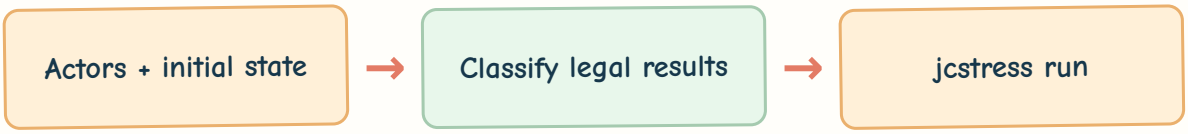
Interview trap: One uncontended microbenchmark cannot predict a high-contention service.

Original source p. 44 · References M22, M31, M32

CHAPTER 15 / 16

Proof, stress testing and evidence

Separate a legal-outcome proof from a finite observation.



FIVE ORIGINAL INTERVIEW QUESTIONS

071. What does jcstress provide?

072. What is the difference between allowed, forbidden and observed outcomes?

073. Why can test coordination accidentally hide a race?

074. Can JMH prove a concurrency protocol correct?

075. What should a concurrency test report include?

Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q071. What does jcstress provide?

SHORT ANSWER · SAY THIS FIRST

jcstress checks whether tiny concurrent tests exhibit allowed, interesting or forbidden outcomes.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

jcstress is an OpenJDK concurrency stress harness for exploring outcomes of small shared-state tests across runtime and hardware conditions. Actors perform the intended concurrent actions and outcome annotations classify observations. It complements a specification-based proof rather than replacing it.

JAVA / CONCURRENCY SCENARIO

```
Actor A: publication actions
Actor B: observation actions
Outcome table: acceptable, interesting or forbidden
```

What happens: The harness increases opportunities to observe challenging executions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Why should the test's accepted outcomes be justified before execution?

Answer: Specify exact code, initial values and expected categories first; observations alone cannot tell you which result violates the JMM.

Interview trap: Passing a finite run is not proof that no legal bad outcome exists.

Original source p. 45 · References M19, M20

Q072 · Proof, stress testing and evidence

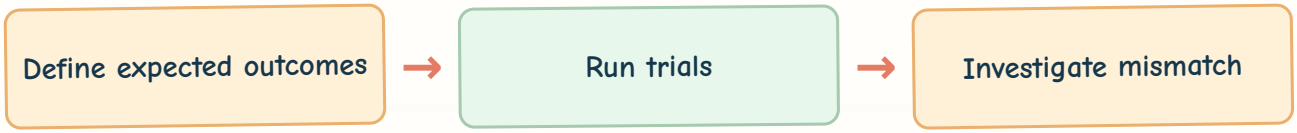
INTERVIEW ANSWER CARD

Q072. What is the difference between allowed, forbidden and observed outcomes?

SHORT ANSWER · SAY THIS FIRST

> Allowed means permitted by the contract; observed means actually seen in a finite run; forbidden should never occur under the model.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Allowed is a contract classification, forbidden conflicts with the stated contract and assumptions, and observed is a finite experiment result. An allowed outcome may be rare or absent on the tested platform. A forbidden observation demands investigation of code, harness, assumptions or implementation.

JAVA / CONCURRENCY SCENARIO

```
Allowed-but-unseen != forbidden
Observed once != all possible executions
Forbidden classification -> explicit proof obligation
```

What happens: Keep the theoretical table separate from the measured histogram.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you investigate an allegedly forbidden result?

Answer: First inspect test code, accidental synchronization, harness setup and the supposed proof. If reproducible, investigate compiler/JVM/platform bugs or a wrong classification.

Interview trap: 'It never failed in a million runs' is an incomplete correctness argument.

Original source p. 46 · References M19, M20

Q073 · Proof, stress testing and evidence

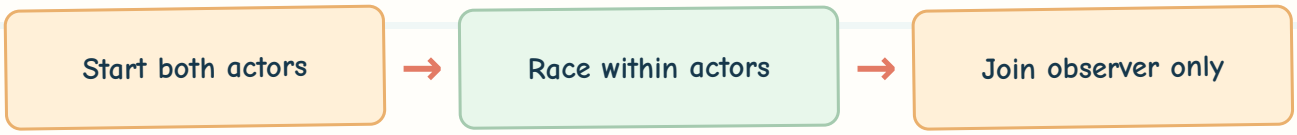
INTERVIEW ANSWER CARD

Q073. Why can test coordination accidentally hide a race?

SHORT ANSWER · SAY THIS FIRST

Test setup can accidentally add a synchronization edge that hides a real race.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

A latch, volatile flag or join placed inside the intended actor interaction can create ordering the production protocol lacks. Use coordination to launch and collect carefully without adding unintended actor-to-actor handoff. State which edges the test introduces.

JAVA / CONCURRENCY SCENARIO

```
Intended race: writer and reader access ordinary fields
Accidental repair: writer countDown -> reader await -> read
Now the test exercises a different synchronization graph.
```

What happens: The test's memory graph is part of its design.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

Which edges does joining actors add to the observer only?

Answer: Joining both actors orders their completed actions before the observer reads final results, but does not retroactively order the actors against each other.

Interview trap: A start gate and a completion observer must not be confused with publication between actor actions.

Original source p. 46 · References M19, M20

Q074 · Proof, stress testing and evidence

INTERVIEW ANSWER CARD

Q074. Can JMH prove a concurrency protocol correct?

SHORT ANSWER · SAY THIS FIRST

JMH measures performance; it is not a memory-model correctness test.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

No. JMH measures performance of a configured benchmark. jcstress explores concurrency outcomes, and reasoning establishes permitted behavior. Use each for its purpose, including representative load tests for application impact.

JAVA / CONCURRENCY SCENARIO

```
JMM/API reasoning -> correctness contract
jcstress -> outcome evidence
JMH -> controlled performance measurement
Service load -> end-to-end operational impact
```

What happens: A benchmark can measure an incorrect protocol very precisely.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you combine these methods for a new lock-free component?

Answer: Prove the protocol and use jcstress for legal outcomes first, then use JMH for microperformance and realistic integration load tests.

Interview trap: Throughput numbers are not a visibility or atomicity proof.

Original source p. 46 · References M19, M20

Q075 · Proof, stress testing and evidence

INTERVIEW ANSWER CARD

Q075. What should a concurrency test report include?

SHORT ANSWER · SAY THIS FIRST

A concurrency report should include code, shared state, allowed outcomes, platform and what was actually observed.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Record source and assumptions, expected outcome categories, runtime/build, architecture, harness configuration and measured observations. Separate deterministic controlled demonstrations from unconstrained race exploration. Include limitations and the reasoning behind rejected outcomes.

JAVA / CONCURRENCY SCENARIO

```
Report: code + initial state + actor actions
Classify: outcomes and synchronization chain
Measure: runtime/platform, runs and actual histogram
Limit: finite observations do not prove universality
```

What happens: A reproducible report lets another engineer challenge the assumptions.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What changes would require rerunning tests after a JDK upgrade?

Answer: Rerun after changes in JVM compiler, runtime flags, architecture, collector, harness version or source because those can change observed interleavings.

Interview trap: Hiding unexpected outcomes as noise weakens the evidence.

Original source p. 47 · References M19, M20

CHAPTER 16 / 16

Senior design and production scenarios

Review lifecycle, failure cleanup and cross-service boundaries.



FIVE ORIGINAL INTERVIEW QUESTIONS

076. Design a consistent runtime configuration service.

077. Design a strict bounded in-memory reservation mechanism.

078. A cache stores thread-safe mappings but requests corrupt values. Why?

079. A stop request is visible but the worker never ends. What next?

080. How do you review a concurrency change as a technical lead?

Interview habit: name the shared variable, writer, reader, synchronization edge, and what happens on failure.

Q076 · Senior design and production scenarios

INTERVIEW ANSWER CARD

Q076. Design a consistent runtime configuration service.

SHORT ANSWER · SAY THIS FIRST

Build one validated immutable config and publish it as one version to every local reader.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Design a consistent runtime configuration service. Build validated immutable snapshots and publish a single version through volatile or an atomic reference. Readers capture one version per operation. If concurrent updates must compose, use a CAS/lock protocol or one writer. Persistence and cross-process propagation are separate concerns.

JAVA / CONCURRENCY SCENARIO

```
record Settings(long version, int timeout, int retries) {}
volatile Settings current;
Reader: Settings s = current; executeUsing(s);
Writer: validate new version; publish whole snapshot;
```

What happens: One reference controls the local publication boundary.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you handle stale versions arriving from a remote event stream?

Answer: Keep a version number; reject stale remote events and use ordered replay/ reconciliation. A local volatile ref cannot agree across replicas.

Interview trap: A JVM volatile field does not propagate changes to other service replicas.

Original source p. 48 · References M08, M13, M15, M34

Q077. Design a strict bounded in-memory reservation mechanism.

SHORT ANSWER · SAY THIS FIRST

A strict local reservation limit needs one coordinated permit per admitted job and exactly one release.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Design a strict bounded in-memory reservation mechanism. Represent the invariant in one atomic transition or critical section. Validate available capacity, reserve it and define exactly when failure/cancellation releases it. Use the matching release policy; for distributed reservations, choose a durable shared authority separately.

JAVA / CONCURRENCY SCENARIO

```
synchronized (lock) {
 if (used == capacity) reject();
 else { used++; reserve(); }
}
```

What happens: The complete local admission decision is protected from concurrent callers.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would an exception during reserve affect used?

Answer: If reserve throws before ownership transfer, release the acquired permit. After transfer, the job completion/cancellation owner must release it once.

Interview trap: Checking a LongAdder or map size before reserving is not a strict atomic capacity decision.

Original source p. 49 · References M08, M13, M15, M34

Q078 · Senior design and production scenarios

INTERVIEW ANSWER CARD

Q078. A cache stores thread-safe mappings but requests corrupt values. Why?

SHORT ANSWER · SAY THIS FIRST

A concurrent map protects entries, but mutable lists or objects stored inside can still race.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

The container may safely coordinate mappings while mutable values remain shared without a policy. Identify who owns and mutates the value graph. Use immutable replacement, guarded values or value-specific concurrency mechanisms with atomic business operations.

JAVA / CONCURRENCY SCENARIO

```
ConcurrentHashMap<Key, ArrayList<Item>> cache;
Two callers -> same list -> uncoordinated mutation
Candidate: immutable list replacement per update
```

What happens: Safety is compositional only when each layer's assumptions align.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would two writers avoid overwriting each other's list additions?

Answer: Use compute for per-key atomic replacement of an immutable copied list, or a lock guarding mutation; do not overwrite another writer's update.

Interview trap: Swapping one concurrent collection for another does not automatically fix every compound update.

Original source p. 49 · References M08, M13, M15, M34

Q079 · Senior design and production scenarios

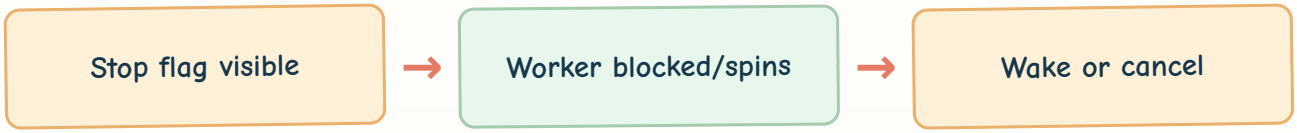
INTERVIEW ANSWER CARD

Q079. A stop request is visible but the worker never ends. What next?

SHORT ANSWER · SAY THIS FIRST

A visible stop flag does not wake blocked I/O or guarantee the worker gets CPU time to finish.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Separate visibility from wakeup and progress. The worker may be blocked, hold a lock, ignore interruption or wait on a dependency. Define cancellation at those boundaries with cleanup and deadlines. Do not replace one memory keyword blindly.

JAVA / CONCURRENCY SCENARIO

```
Volatile stop flag becomes true
Worker waits in blocking I/O without checking it
Need: supported wakeup/cancellation plus cleanup policy
```

What happens: The visibility proof can be correct while the termination protocol is incomplete.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

How would you distinguish a waiting worker from a CPU spin loop?

Answer: Capture thread dumps and CPU samples: WAITING/BLOCKED with stack frames suggests a wait, whereas repeated RUNNABLE samples with high CPU suggests a spin.

Interview trap: A synchronized stop flag cannot force arbitrary blocking work to return.

Original source p. 50 · References M08, M13, M15, M34

Q080 · Senior design and production scenarios

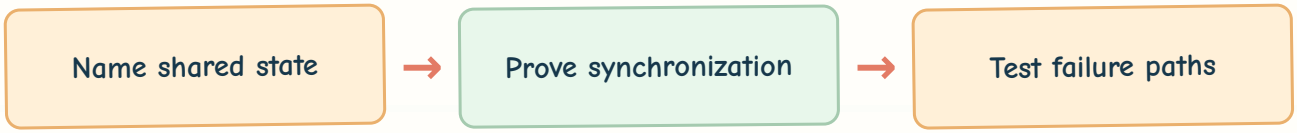
INTERVIEW ANSWER CARD

Q080. How do you review a concurrency change as a technical lead?

SHORT ANSWER · SAY THIS FIRST

> Review who owns shared data, every operation's invariant, synchronization edges and exceptional cleanup.

THREAD / MEMORY FLOW DIAGRAM



PRECISE TECHNICAL EXPLANATION

Identify shared state, ownership, the invariant and every read/write path. Draw publication and mutation edges, review progress/failure behavior, then combine proof with targeted tests and measurements. Check distributed boundaries separately from JVM guarantees.

JAVA / CONCURRENCY SCENARIO

```
State -> owner -> synchronization -> invariant
Lifecycle -> cancellation -> failure cleanup
Evidence -> stress outcomes -> performance -> rollout
```

What happens: A review should explain why the design remains correct during exceptional paths and future maintenance.

ANSWERED FOLLOW-UP · PRACTICE ALOUD

What documentation would keep future changes from bypassing the locking policy?

Answer: Write a short shared-state contract beside the API: guarded fields, lock identity, allowed access paths, lifecycle transitions and mandatory race tests for changes.

> **Interview trap:** JMM guarantees do not make database, message or cross-service transactions atomic.

Original source p. 50 · References M08, M13, M15, M34

ORIGINAL GUIDE · REFERENCE P. 51

Mechanism selection: name the invariant

Choose the synchronization tool from the actual correctness need.

```
invariant
NEED
STARTING CHOICE
BOUNDARY
One-shot initialized
payload
Volatile flag/reference
Later mutation and reuse need another
protocol.
Exact concurrent
count
AtomicInteger / AtomicLong
Separate checks and updates still
compose poorly.
Multi-field state
transition
One lock or atomic immutable
snapshot
All access paths must respect the same
policy.
Work ownership
transfer
BlockingQueue
Producer must not mutate transferred
state unsafely.
Task completion result
Future.get
Timeout is not successful completion.
High-update statistics
LongAdder
sum is not an atomic concurrent
snapshot.
Guarded waiting
predicate
Condition or monitor loop
Wakeup does not itself prove the
predicate.
Cross-service
reservation
Shared durable authority
A JVM keyword cannot coordinate
replicas.
References: M04, M08-M17, M24, M34
```

> This is a study reference, not a complete distributed or production-safe implementation.

ORIGINAL GUIDE · REFERENCE P. 52

Litmus outcome reference

Know which outcomes Java allows and which the test forbids.

```
Fresh state per experiment; no other writes or synchronization between actors. The
observer collects
results after actor completion. Allowed outcomes are contract classifications, not
measured
frequencies.
TEST
ALLOWED
FORBIDDEN HERE
Plain message passing
-1 (not ready), 0, 42
Other payload values not justified
by this program.
Volatile one-shot flag
-1 or 42
0 after reading true.
Plain store buffering
(0,0), (0,1), (1,0), (1,1)
Do not invent writes beyond initial
0 and written 1.
Both locations volatile
(0,1), (1,0), (1,1)
(0,0).
Correctly constructed final
x=1, ordinary y=2
Non-null object: x=1; ordinary y
need not be 2
Treating final as general
publication of y.
Plain message passing: The flag is ordinary. Observing ready does not acquire
initialization of
data. The reader can skip the payload read or see its initial/written value.
Volatile message passing: The true read acquires the one-shot publication. Payload
write ->
volatile write -> volatile read -> payload read.
Volatile store buffering: Assume both reads saw zero. Each read must precede the
other thread's
write in synchronization order, while program order requires each own write before
its read. The
resulting cycle cannot be ordered.
References: M01, M02, M19-M23
```

> This is a study reference, not a complete distributed or production-safe implementation.

ORIGINAL GUIDE · REFERENCE P. 53

Worked review: fractured configuration

One immutable snapshot avoids mixed-version field reads.

```
A service stores timeout and retries in two separately volatile fields. Writers
update both; requests
read both independently. The access operations are ordered, but one request can
combine values
from different updates.
// Bad invariant representation: two independent locations
volatile int timeout = 1000;
volatile int retries = 2;
// Better: one validated, immutable version
record Config(long version, int timeout, int retries) {}
volatile Config current = new Config(1, 1000, 2);
Config c = current; // capture exactly once for this operation
Invariant: Every request uses a configuration combination that was validated as one
version.
Publication: Writer completes construction, then assigns current. Reader obtains one
published
reference and keeps it locally.
Concurrent writers: Whole-reference assignment alone can lose logical updates.
Serialize writers,
apply version rejection or use CAS with side-effect-free recomputation.
Distributed boundary: Define authoritative versions and stale-event handling between
replicas.
Local volatile publication does not establish cluster agreement.
References: M05, M15, M21
```

> This is a study reference, not a complete distributed or production-safe implementation.

ORIGINAL GUIDE · REFERENCE P. 54

Worked review: reservation and cleanup

Admission and permit release need one coordinated lifecycle.

```
A strict local limit of N jobs requires admission, ownership and release to describe
the same lifecycle.
A map-size check or statistics counter cannot enforce that lifecycle by itself.
if (!permits.tryAcquire()) return REJECTED;
try {
    // Transfer the permit to the admitted job's lifecycle.
    // Arrange exactly one release on completion/cancellation.
    submitOwnedJob();
} catch (RuntimeException failure) {
    permits.release(); // submission failed before ownership transfer
    throw failure;
}
Design boundary: This is protocol notation, not a complete executor wrapper. Define
the exact
point at which submitOwnedJob transfers responsibility and ensure exceptions cannot
trigger two
releases.
Failure questions: What if submission rejects? What if cancellation arrives before
execution? Can
both callback and finally release? Does completion include downstream work or only
enqueueing?
Proof: Count acquired permits not yet released. Every admission consumes one, every
terminal
lifecycle releases once, and unsuccessful admission consumes none.
Evidence: Test rejection, immediate completion, queued cancellation, exceptions and
concurrent
completion/cancel. Check the invariant, not only final totals.
References: M12, M28
```

> This is a study reference, not a complete distributed or production-safe implementation.

HANDS-ON PRACTICE

8 runnable Java exercises

Predict the outcome. Run the program. Explain the JMM proof.

Compile --release 17 → Run chosen class → Prove its edge

Requirements: Full JDK 17+ and Bash; all complete programs are independent of Maven and Spring Boot.

Verification: Each of the original eight Java sources compiled under --release 17 and produced its expected output during this build.

A finite run is evidence only. It does not prove that every possible race schedule has been tested.

HANDS-ON EXERCISE 01 / 08

L01. One-shot volatile publication

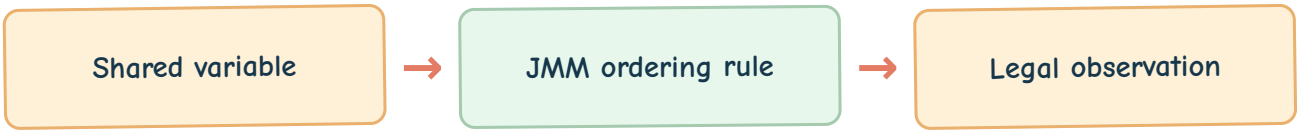
The payload read happens before join; the volatile flag is the publication edge.

COMMANDS

```
javac --release 17 *.java
java Publication
```

Expected console output:
`publication=42`

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 02 / 08

L02. Controlled volatile lost update

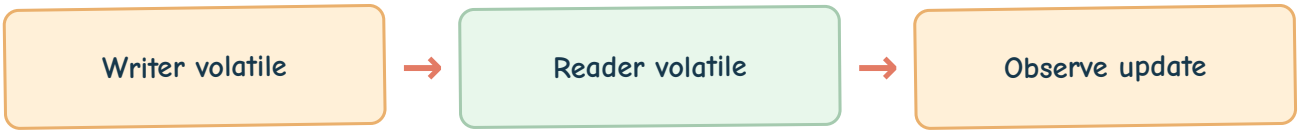
Both increments deliberately read the same old count before writing.

COMMANDS

```
javac --release 17 *.java
java LostUpdate
```

Expected console output:
`two increments, count=1`

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 03 / 08

L03. Exact atomic increments

Four threads each increment 10,000 times using AtomicInteger.

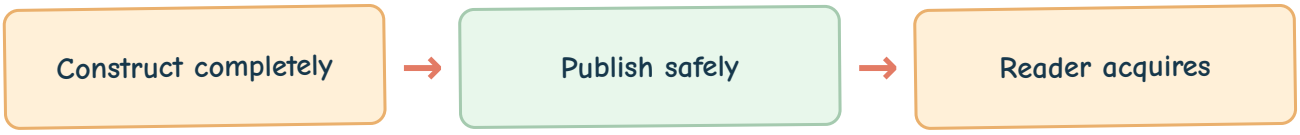
COMMANDS

```
javac --release 17 *.java
java AtomicCounter
```

Expected console output:

```
atomic count=40000
```

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 04 / 08

L04. Whole immutable configuration snapshot

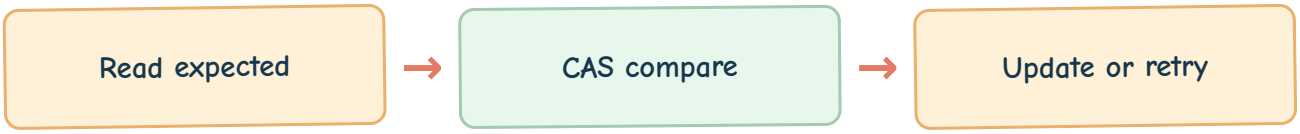
A request captures one immutable configuration snapshot.

COMMANDS

```
javac --release 17 *.java
java ConfigSnapshot
```

Expected console output:
`snapshot invariant preserved`

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 05 / 08

L05. Executor input and completion result

Executor submission and successful Future.get create two handoffs.

COMMANDS

```
javac --release 17 *.java
java FutureHandoff
```

Expected console output:

```
future result=42
```

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 06 / 08

L06. Ownership transfer through a queue

The producer finishes the message before transferring ownership through the queue.

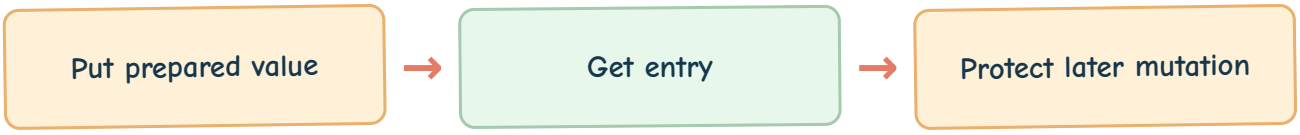
COMMANDS

```
javac --release 17 *.java
java QueueHandoff
```

Expected console output:

```
queue payload=42
```

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 07 / 08

L07. Publish before countDown

The countDown comes after the result write; await reads only on success.

COMMANDS

```
javac --release 17 *.java
java LatchHandoff
```

Expected console output:

```
latch result=42
```

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

HANDS-ON EXERCISE 08 / 08

L08. VarHandle publication modes

The reader observes a release write through an acquire read on the same variable.

COMMANDS

```
javac --release 17 *.java
java ReleaseAcquire
```

Expected console output:
`release/acquire payload=42`

Interview proof: Explain the exact atomic action or publication edge. State what changes if the ordering or lifecycle assumption is removed.



Lab limit: The outcome check is not an exhaustive memory-model correctness proof. The actual source is included in the following pages and downloadable ZIP.

RUNNABLE APPENDIX

Eight complete Java files

The following code is preserved from the supplied PDF. Run each class separately after compiling all sources.

Save original .java → Compile on JDK 17+ → Run & explain result

These runnable examples deliberately focus on one concurrency property and include exact result checks. They are not benchmark suites or substitutes for the jcstress harness.

Do not change the synchronization while assuming the expected output and forbidden outcomes remain the same.

SOURCE APPENDIX · COMPILED

Publication.java

```
public class Publication {
  int data;
  volatile boolean ready;
  public static void main(String[] args) throws Exception {
    Publication s = new Publication();
    Thread writer = new Thread(() -> { s.data = 42; s.ready = true; });
    writer.start();
    long deadline = System.nanoTime() + 5_000_000_000L;
    while (!s.ready && System.nanoTime() < deadline) Thread.onSpinWait();
    if (!s.ready || s.data != 42) throw new AssertionError();
    writer.join();
    System.out.println("publication=42");
  }
}
```

Run: `javac --release 17 *.java` then `java Publication`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

LostUpdate.java

```
import java.util.concurrent.CyclicBarrier;
public class LostUpdate {
  volatile int count;
  final CyclicBarrier gate = new CyclicBarrier(2);
  void incrementInSteps() {
    int old = count;
    try { gate.await(); }
    catch (Exception e) { throw new RuntimeException(e); }
    count = old + 1;
  }
  public static void main(String[] args) throws Exception {
    LostUpdate s = new LostUpdate();
    Thread a = new Thread(s::incrementInSteps);
    Thread b = new Thread(s::incrementInSteps);
    a.start(); b.start(); a.join(); b.join();
    if (s.count != 1) throw new AssertionError(s.count);
    System.out.println("two increments, count=1");
  }
}
```

Run: `javac --release 17 *.java` then `java LostUpdate`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

AtomicCounter.java

```
import java.util.concurrent.atomic.AtomicInteger;
public class AtomicCounter {
  public static void main(String[] args) throws Exception {
    AtomicInteger count = new AtomicInteger();
    Thread[] threads = new Thread[4];
    for (int t = 0; t < threads.length; t++) {
      threads[t] = new Thread(() -> {
        for (int i = 0; i < 10_000; i++) count.incrementAndGet();
      });
      threads[t].start();
    }
    for (Thread thread : threads) thread.join();
    if (count.get() != 40_000) throw new AssertionError();
    System.out.println("atomic count=40000");
  }
}
```

Run: `javac --release 17 *.java` then `java AtomicCounter`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

ConfigSnapshot.java

```
import java.util.concurrent.atomic.AtomicInteger;
public class ConfigSnapshot {
  record Config(int version, int doubled) {}
  volatile Config current = new Config(0, 0);
  public static void main(String[] args) throws Exception {
    ConfigSnapshot s = new ConfigSnapshot();
    AtomicInteger errors = new AtomicInteger();
    Thread writer = new Thread(() -> {
      for (int i = 1; i <= 100_000; i++)
        s.current = new Config(i, i * 2);
    });
    Thread reader = new Thread(() -> {
      for (int i = 0; i < 100_000; i++) {
        Config c = s.current;
        if (c.doubled() != c.version() * 2) errors.incrementAndGet();
      }
    });
    writer.start(); reader.start(); writer.join(); reader.join();
    if (errors.get() != 0) throw new AssertionError();
    System.out.println("snapshot invariant preserved");
  }
}
```

Run: `javac --release 17 *.java` then `java ConfigSnapshot`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

FutureHandoff.java

```
import java.util.concurrent.*;
public class FutureHandoff {
  int input;
  int result;
  public static void main(String[] args) throws Exception {
    FutureHandoff s = new FutureHandoff();
    ExecutorService pool = Executors.newSingleThreadExecutor();
    try {
      s.input = 21;
      Future<?> task = pool.submit(() -> s.result = s.input * 2);
      task.get(5, TimeUnit.SECONDS);
      if (s.result != 42) throw new AssertionError();
      System.out.println("future result=42");
    } finally { pool.shutdownNow(); }
  }
}
```

Run: `javac --release 17 *.java` then `java FutureHandoff`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

QueueHandoff.java

```
import java.util.concurrent.*;
public class QueueHandoff {
  static class Message { int data; }
  public static void main(String[] args) throws Exception {
    BlockingQueue<Message> queue = new ArrayBlockingQueue<>(1);
    Thread producer = new Thread(() -> {
      Message message = new Message(); message.data = 42;
      try { queue.put(message); }
      catch (InterruptedException e) { Thread.currentThread().interrupt(); }
      // No producer mutation after transfer.
    });
    producer.start();
    Message received = queue.poll(5, TimeUnit.SECONDS);
    if (received == null || received.data != 42) throw new AssertionError();
    producer.join();
    System.out.println("queue payload=42");
  }
}
```

Run: `javac --release 17 *.java` then `java QueueHandoff`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

LatchHandoff.java

```
import java.util.concurrent.*;
public class LatchHandoff {
  int result;
  public static void main(String[] args) throws Exception {
    LatchHandoff s = new LatchHandoff();
    CountDownLatch done = new CountDownLatch(1);
    Thread worker = new Thread(() -> { s.result = 42; done.countDown(); });
    worker.start();
    if (!done.await(5, TimeUnit.SECONDS)) throw new AssertionError("timeout");
    if (s.result != 42) throw new AssertionError();
    worker.join();
    System.out.println("latch result=42");
  }
}
```

Run: `javac --release 17 *.java` then `java LatchHandoff`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

SOURCE APPENDIX · COMPILED

ReleaseAcquire.java

```
import java.lang.invoke.*;
public class ReleaseAcquire {
  int payload;
  boolean ready;
  static final VarHandle READY;
  static {
    try {
      READY = MethodHandles.lookup().findVarHandle(
          ReleaseAcquire.class, "ready", boolean.class);
    } catch (ReflectiveOperationException e) {
      throw new ExceptionInInitializerError(e);
    }
  }
  public static void main(String[] args) throws Exception {
    ReleaseAcquire s = new ReleaseAcquire();
    Thread writer = new Thread(() -> {
      s.payload = 42; READY.setRelease(s, true);
    });
    writer.start();
    long deadline = System.nanoTime() + 5_000_000_000L;
    boolean acquired;
    do {
      acquired = (boolean) READY.getAcquire(s);
      if (acquired) break;
      Thread.onSpinWait();
    } while (System.nanoTime() < deadline);
    if (!acquired || s.payload != 42) throw new AssertionError();
    writer.join();
    System.out.println("release/acquire payload=42");
  }
}
```

Run: `javac --release 17 *.java` then `java ReleaseAcquire`

Read the code guards, ownership and expected result. A passing run demonstrates this test, not every interleaving.

OPTIONAL STRESS-TESTING TEMPLATE

VolatilePublicationTest · jcstress

```
package org.sample;
import org.openjdk.jcstress.annotations.*;
import org.openjdk.jcstress.infra.results.I_Result;
@JCStressTest
@Outcome(id = "-1", expect = Expect.ACCEPTABLE,
         desc = "Publication not observed")
@Outcome(id = "42", expect = Expect.ACCEPTABLE,
         desc = "Initialized payload observed")
@Outcome(expect = Expect.FORBIDDEN,
         desc = "Other result under one-shot assumptions")
@State
public class VolatilePublicationTest {
  int data;
  volatile boolean ready;
  @Actor public void writer() { data = 42; ready = true; }
  @Actor public void reader(I_Result r) {
    r.r1 = ready ? data : -1;
  }
}
```

Expected: -1 and 42 are acceptable; any other result is forbidden under the stated one-shot assumptions.

Requires a real jcstress Maven/archetype project. Template was not compiled or executed; use an available supported dependency version.

DECISION REFERENCE

Which tool should I choose?

One-shot payload: Volatile flag/reference; no uncontrolled reuse.

Exact single counter: AtomicInteger or AtomicLong for one value.

Multi-field business state: One lock or CAS over an immutable snapshot.

Worker completion: Future.get or CountDownLatch successful await.

Queued work ownership: BlockingQueue with immutable or transferred messages.

High-update statistics: LongAdder; sum is not a live atomic snapshot.

Waiting condition: Condition/monitor loop with guarded predicate.

Strict admission: Semaphore plus exactly-once permit release.

Cross-service inventory: Shared durable authority; Java memory ordering is local.

INTERVIEW REVISION

The five proof questions

1. Shared state
Which fields or objects can two threads touch?

2. Ownership
Who writes and who may read at each lifecycle state?

3. Synchronization
What exact volatile, lock, CAS or library edge connects them?

4. Invariant
Which bad combination must never become visible?

5. Failure/progress
Can timeout, cancel, rejection or exception break cleanup?

Strong senior answer: draw a three-step ordering chain and then identify one unprotected failure path.

14-DAY STUDY SPRINT

Prepare it in 14 days

Day 01. JMM terms: visibility, atomicity, ordering and data races.

Day 02. Happens-before: draw publication chains.

Day 03. Volatile: run L01 and L02; explain lost updates.

Day 04. Locks, wait/notify and condition loops.

Day 05. Safe publication: DCL and holder singleton.

Day 06. Final fields, escaping this and deep immutability.

Day 07. CAS/AtomicInteger; run L03 and design immutable Range.

Day 08. VarHandle release/acquire; run L08.

Day 09. Cancellation, wakeup and liveness.

Day 10. Executors, BlockingQueue and latch; run L05-L07.

Day 11. Application snapshots and Spring singleton; run L04.

Day 12. Predict legal outcomes of MP and store buffering.

Day 13. Write a jcstress report and separate JMH performance.

Day 14. Mock interview: 10 random Qs with invariants and failure cases.

SOURCE REFERENCE DIRECTORY

Primary Java Memory Model sources

These links are preserved from the original guide. Consult the JLS, JSR-133 and documented Java concurrency APIs for precise memory-consistency effects.

https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html

https://docs.oracle.com/javase/specs/jls/se25/html/jls-17.html#jls-17.5

https://docs.oracle.com/javase/specs/jls/se25/html/jls-12.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/package-summary.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/invoke/VarHandle.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/package-summary.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/locks/Lock.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/locks/Condition.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CountDownLatch.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Future.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/BlockingQueue.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/AtomicReference.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/AtomicInteger.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/LongAdder.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/locks/StampedLock.html

https://github.com/openjdk/jcstress

https://github.com/openjdk/jcstress/tree/master/jcstress-samples/src/main/java/org/openjdk/jcstress/samples

https://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html

https://gee.cs.oswego.edu/dl/jmm/cookbook.html

https://www.cs.umd.edu/~pugh/java/memoryModel/jsr133.pdf

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/atomic/AtomicStampedReference.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CompletableFuture.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CyclicBarrier.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Semaphore.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ThreadLocal.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CopyOnWriteArrayList.html

https://openjdk.org/jeps/444

https://openjdk.org/jeps/491

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/locks/LockSupport.html

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/locks/ReentrantLock.html

> The source specifies legal outcomes; local tests only report what was observed. Java 21/25 discussion is not a claim of having executed those runtime variations.